



[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

Index

Nvim `:help` pages, [generated](#) from [source](#) using the [tree-sitter-vimdoc](#) parser.

This file contains a list of all commands for each mode, with a tag and a short description. The lists are sorted on ASCII value.

Tip: When looking for certain functionality, use a search command. E.g., to look for deleting something, use: `/delete`.

For an overview of options see [option-list](#).

For an overview of built-in functions see [functions](#).

For a list of Vim variables see [vim-variable](#).

1. Insert mode

[insert-index](#)

Tag	Char	Insert-mode
action		

i_CTRL-@	CTRL-@	insert
previously inserted text and stop		
		insert
i_CTRL-A	CTRL-A	insert
previously inserted text		
i_CTRL-C	CTRL-C	quit insert
mode, without checking for		
		abbreviation
i_CTRL-D	CTRL-D	delete one
shiftwidth of indent in the current		
		line
i_CTRL-E	CTRL-E	insert the

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

i_CTRL-E	CTRL-E	insert the character which is below the cursor
i_CTRL-F	CTRL-F	not used (but by default it's in 'cinkeys' to re-indent the current line)
i_CTRL-G_j	CTRL-G CTRL-J	line down, to column where inserting started
i_CTRL-G_j	CTRL-G j	line down, to column where inserting started
i_CTRL-G_j	CTRL-G <Down>	line down, to column where inserting started
i_CTRL-G_k	CTRL-G CTRL-K	line up, to column where inserting started
i_CTRL-G_k	CTRL-G k	line up, to column where inserting started
i_CTRL-G_k	CTRL-G <Up>	line up, to column where inserting started
i_CTRL-G_u	CTRL-G u	start new undoable edit
i_CTRL-G_U	CTRL-G U	don't break undo with next cursor movement
i_<BS>	<BS>	delete character before the cursor
i_digraph	{char1}<BS>{char2}	enter digraph (only when 'digraph' option set)
i_CTRL-H	CTRL-H	same as <BS>
i_<Tab>	<Tab>	insert a <Tab> character
i_CTRL-I	CTRL-I	same as <Tab>
i_<NL>	<NL>	same as <CR>
i_CTRL-J	CTRL-J	same as <CR>
i_CTRL-K	CTRL-K {char1} {char2}	enter digraph
i_<CR>	<CR>	begin new line
i_CTRL-M	CTRL-M	same as <CR>
i_CTRL-N	CTRL-N	find next match for keyword in front of the cursor
i_CTRL-O	CTRL-O	execute a

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

[i_CTRL-O](#) [CTRL-O](#) execute a single command and return to insert mode

[i_CTRL-P](#) [CTRL-P](#) find previous match for keyword in front of the cursor

[i_CTRL-Q](#) [CTRL-Q](#) same as [CTRL-V](#), unless used for terminal control flow

[i_CTRL-SHIFT-Q](#) [CTRL-SHIFT-Q](#) [{char}](#) like [CTRL-Q](#) unless [tui-modifyOtherKeys](#) is active

[i_CTRL-R](#) [CTRL-R](#) [{register}](#) insert the contents of a register

[i_CTRL-R_CTRL-R](#) [CTRL-R](#) [CTRL-R](#) [{register}](#) insert the contents of a register literally

[i_CTRL-R_CTRL-O](#) [CTRL-R](#) [CTRL-O](#) [{register}](#) insert the contents of a register literally and don't auto-indent

[i_CTRL-R_CTRL-P](#) [CTRL-R](#) [CTRL-P](#) [{register}](#) insert the contents of a register literally and fix indent.

[CTRL-S](#) not used or used for terminal control flow

[i_CTRL-T](#) [CTRL-T](#) insert one shiftwidth of indent in current line

[i_CTRL-U](#) [CTRL-U](#) delete all entered characters in the current line

[i_CTRL-V](#) [CTRL-V](#) [{char}](#) insert next non-digit literally

[i_CTRL-SHIFT-V](#) [CTRL-SHIFT-V](#) [{char}](#) like [CTRL-V](#) unless [tui-modifyOtherKeys](#) is active

[i_CTRL-V_digit](#) [CTRL-V](#) [{number}](#) insert three

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

[i_CTRL-V_digit_CTRL-V {number}](#) insert three digit decimal number as a single byte.

[i_CTRL-W](#) [CTRL-W](#) delete word before the cursor

[i_CTRL-X](#) [CTRL-X {mode}](#) enter [CTRL-X](#) sub mode, see [i_CTRL-X_index](#)

[i_CTRL-Y](#) [CTRL-Y](#) insert the character which is above the cursor

[i_<Esc>](#) [<Esc>](#) end insert mode

[i_CTRL-\[](#) [CTRL-\[](#) same as [<Esc>](#)

[i_CTRL-_CTRL-N](#) [CTRL-\](#) [CTRL-N](#) go to Normal mode

[i_CTRL-_CTRL-G](#) [CTRL-\](#) [CTRL-G](#) go to Normal mode

[CTRL-\ a - z](#) reserved for extensions

[CTRL-\ others](#) not used

[i_CTRL-\]](#) [CTRL-\]](#) trigger abbreviation

[i_CTRL-^](#) [CTRL-^](#) toggle use of [:lmap](#) mappings

[i_CTRL-_](#) [CTRL-_](#) When ['allowrevins'](#) set: toggle ['revins'](#)

[<Space>](#) to ['~'](#) not used, except ['0'](#) and ['^'](#) followed by

[CTRL-D](#)

[i_0_CTRL-D](#) [0](#) [CTRL-D](#) delete all indent in the current line

[i_^_CTRL-D](#) [^](#) [CTRL-D](#) delete all indent in the current line, restore it in the next line

[i_](#) [](#) delete character under the cursor

Meta characters (0x80 to 0xff, 128 to 255)

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

		not used	Main
i_<Left>	<Left>	cursor one	Commands index
character left			Quick reference
i_<S-Left>	<S-Left>	cursor one word	
left			1. Insert Mode
i_<C-Left>	<C-Left>	cursor one word	2. Normal Mode
left			2.1 Text Objects
i_<Right>	<Right>	cursor one	2.2 Window Commands
character right			2.3 Square Bracket Com
i_<S-Right>	<S-Right>	cursor one word	2.4 Commands Starting
right			2.5 Commands Starting
i_<C-Right>	<C-Right>	cursor one word	2.6 Operator-Pending M
right			3. Visual Mode
i_<Up>	<Up>	cursor one line	4. Command-Line Editin
up			5. Terminal Mode
i_<S-Up>	<S-Up>	same as	6. Ex Commands
<PageUp>			
i_<Down>	<Down>	cursor one line	
down			
i_<S-Down>	<S-Down>	same as	
<PageDown>			
i_<Home>	<Home>	cursor to start	
of line			
i_<C-Home>	<C-Home>	cursor to start	
of file			
i_<End>	<End>	cursor past end	
of line			
i_<C-End>	<C-End>	cursor past end	
of file			
i_<PageUp>	<PageUp>	one screenful	
backward			
i_<PageDown>	<PageDown>	one screenful	
forward			
i_<F1>	<F1>	same as <Help>	
i_<Help>	<Help>	stop insert	
mode and display help window			
i_<Insert>	<Insert>	toggle Insert/	
Replace mode			
i_<LeftMouse>	<LeftMouse>	cursor at mouse	

CLICK

[i_<ScrollWheelDown>](#)
[<ScrollWheelDown>](#) move window three lines
 down
[i_<S-ScrollWheelDown>](#) [<S-](#)
[ScrollWheelDown>](#) move window one page
 down
[i_<ScrollWheelUp>](#)
[<ScrollWheelUp>](#) move window three lines
 up
[i_<S-ScrollWheelUp>](#) [<S-](#)
[ScrollWheelUp>](#) move window one page up
[i_<ScrollWheelLeft>](#)
[<ScrollWheelLeft>](#) move window six columns
 left
[i_<S-ScrollWheelLeft>](#) [<S-](#)
[ScrollWheelLeft>](#) move window one page
 left
[i_<ScrollWheelRight>](#)
[<ScrollWheelRight>](#) move window six columns
 right
[i_<S-ScrollWheelRight>](#) [<S-](#)
[ScrollWheelRight>](#) move window one page
 right

commands in [CTRL-X](#)

submode [i_CTRL-X_index](#)

[i_CTRL-X_CTRL-D](#) [CTRL-X](#) [CTRL-D](#)
 complete defined identifiers
[i_CTRL-X_CTRL-E](#) [CTRL-X](#) [CTRL-E](#) scroll
 up
[i_CTRL-X_CTRL-F](#) [CTRL-X](#) [CTRL-F](#)
 complete file names
[i_CTRL-X_CTRL-I](#) [CTRL-X](#) [CTRL-I](#)
 complete identifiers
[i_CTRL-X_CTRL-K](#) [CTRL-X](#) [CTRL-K](#)
 complete identifiers from dictionary
[i_CTRL-X_CTRL-L](#) [CTRL-X](#) [CTRL-L](#)
 complete whole lines
[i_CTRL-X_CTRL-N](#) [CTRL-X](#) [CTRL-N](#) next

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

completion			
i_CTRL-X_CTRL-O	CTRL-X	CTRL-O	omni
completion			
i_CTRL-X_CTRL-P	CTRL-X	CTRL-P	
previous completion			
i_CTRL-X_CTRL-R	CTRL-X	CTRL-R	
complete contents from registers			
i_CTRL-X_CTRL-S	CTRL-X	CTRL-S	
spelling suggestions			
i_CTRL-X_CTRL-T	CTRL-X	CTRL-T	
complete identifiers from thesaurus			
i_CTRL-X_CTRL-Y	CTRL-X	CTRL-Y	scroll
down			
i_CTRL-X_CTRL-U	CTRL-X	CTRL-U	
complete with <code>'completefunc'</code>			
i_CTRL-X_CTRL-V	CTRL-X	CTRL-V	
complete like in <code>:</code> command line			
i_CTRL-X_CTRL-Z	CTRL-X	CTRL-Z	stop
completion, text is unchanged			
i_CTRL-X_CTRL-]	CTRL-X	CTRL-]	
complete tags			
i_CTRL-X_s	CTRL-X	<code>s</code>	
spelling suggestions			

commands in completion mode (see [popupmenu-keys](#))

[complete_CTRL-E](#) [CTRL-E](#) stop completion and go
back to original text

[complete_CTRL-Y](#) [CTRL-Y](#) accept selected match
and stop completion

[CTRL-L](#) insert one
character from the current match

[<CR>](#) insert
currently selected match

[<BS>](#) delete one
character and redo search

[CTRL-H](#) same as [<BS>](#)

[<Up>](#) select the

previous match

	<code><Down></code>	select the next
match		
	<code><PageUp></code>	select a match
several entries back		
	<code><PageDown></code>	select a match
several entries forward		
other		stop completion
and insert the typed character		

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

2. Normal mode

[normal-index](#)

CHAR any non-blank character

WORD a sequence of non-blank characters

N a number entered before the command

`{motion}` a cursor movement command

Nmove the text that is moved over with a `{motion}`

SECTION a section that possibly starts with `'}`' instead of `'{'`

note: 1 = cursor movement command; 2 = can be undone/redone

Tag	Char	Note	Normal-mode
action			

	<code>CTRL-@</code>		not used
CTRL-A	<code>CTRL-A</code>	2	add N to
number at/after cursor			
CTRL-B	<code>CTRL-B</code>	1	scroll N
screens Backwards			
CTRL-C	<code>CTRL-C</code>		interrupt
current (search) command			
CTRL-D	<code>CTRL-D</code>		scroll Down
N lines (default: half a screen)			
CTRL-E	<code>CTRL-E</code>		scroll N
lines upwards (N lines Extra)			
CTRL-F	<code>CTRL-F</code>	1	scroll N

screens Forward				Main
CTRL-G	CTRL-G		display	Commands index
current file name and position				Quick reference
<BS>	<BS>	1	same as "h"	
CTRL-H	CTRL-H	1	same as "h"	
<Tab>	<Tab>	1	go to N	1. Insert Mode
newer entry in jump list				2. Normal Mode
CTRL-I	CTRL-I	1	same as	2.1 Text Objects
<Tab>				2.2 Window Commands
<NL>	<NL>	1	same as "j"	2.3 Square Bracket Com
<S-NL>	<S-NL>	1	same as	2.4 Commands Starting
CTRL-F				2.5 Commands Starting
CTRL-J	CTRL-J	1	same as "j"	2.6 Operator-Pending M
	CTRL-K		not used	3. Visual Mode
CTRL-L	CTRL-L		redraw	4. Command-Line Editin
screen				5. Terminal Mode
<CR>	<CR>	1	cursor to	6. Ex Commands
the first CHAR N lines lower				
<S-CR>	<S-CR>	1	same as	
CTRL-F				
CTRL-M	CTRL-M	1	same as <CR>	
CTRL-N	CTRL-N	1	same as "j"	
CTRL-O	CTRL-O	1	go to N	
older entry in jump list				
CTRL-P	CTRL-P	1	same as "k"	
	CTRL-Q		not used, or	
used for terminal control flow				
CTRL-R	CTRL-R	2	redo changes	
which were undone with 'u'				
	CTRL-S		not used, or	
used for terminal control flow				
CTRL-T	CTRL-T		jump to N	
older Tag in tag list				
CTRL-U	CTRL-U		scroll N	
lines Upwards (default: half a				
			screen)	
CTRL-V	CTRL-V		start	
blockwise Visual mode				
CTRL-W	CTRL-W {char}		window	
commands, see CTRL-W				

CTRL-X	CTRL-X	2	subtract N	Main
from number at/after cursor				Commands index
CTRL-Y	CTRL-Y		scroll N	Quick reference
lines downwards				
CTRL-Z	CTRL-Z		suspend	
program (or start new shell)				1. Insert Mode
	CTRL-[<Esc>		not used	2. Normal Mode
CTRL-_CTRL-N	CTRL-_CTRL-N		go to Normal	2.1 Text Objects
mode (no-op)				2.2 Window Commands
CTRL-_CTRL-G	CTRL-_CTRL-G		go to Normal	2.3 Square Bracket Com
mode (no-op)				2.4 Commands Starting
	CTRL-\ a - z		reserved for	2.5 Commands Starting
extensions				2.6 Operator-Pending M
	CTRL-\ others		not used	3. Visual Mode
CTRL-]	CTRL-]		:ta to ident	4. Command-Line Editin
under cursor				5. Terminal Mode
CTRL-^	CTRL-^		edit Nth	6. Ex Commands
alternate file (equivalent to				
			":e #N")	
CTRL-<Tab>	CTRL-<Tab>		same as	
g<Tab> : go to last accessed tab				
			page	
	CTRL-_		not used	
<Space>	<Space>	1	same as "l"	
!	!{motion}{filter}			
		2	filter Nmove	
text through the {filter}				
			command	
!!	!!{filter}	2	filter N	
lines through the {filter} command				
quote	"{register}		use	
{register} for next delete, yank or put				
			({.%#:.} only	
work with put)				
#	#	1	search	
backward for the Nth occurrence of				
			the ident	
under the cursor				
\$	\$	1	cursor to	

the end of Nth next line

[%](#) [%](#) 1 find the
next (curly/square) bracket on
this line

and go to its match, or go to
matching

comment bracket, or go to matching
preprocessor

directive.

[N%](#) [{count}%](#) 1 go to N
percentage in the file

[&](#) [&](#) 2 repeat last

[:s](#)

['](#) ['{a-zA-Z0-9}](#) 1 cursor to
the first CHAR on the line with
mark [{a-zA-](#)

[Z0-9}](#)

[''](#) 1 cursor to
the first CHAR of the line where
the cursor

was before the latest jump.

['\('](#) 1 cursor to
the first CHAR on the line of the
start of the

current sentence

['\)'](#) 1 cursor to
the first CHAR on the line of the
end of the

current sentence

['<](#) 1 cursor to
the first CHAR of the line where
highlighted

area starts/started in the
current

buffer.

['>](#) 1 cursor to
the first CHAR of the line where
highlighted

area ends/ended in the current
buffer.

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

'L	'L	1	cursor to the first CHAR on the line of the last operated text or start of put text	Main
				Commands index
				Quick reference
']	']	1	cursor to the first CHAR on the line of the operated text or end of put text	1. Insert Mode
				2. Normal Mode
				2.1 Text Objects
				2.2 Window Commands
				2.3 Square Bracket Com
				2.4 Commands Starting
				2.5 Commands Starting
				2.6 Operator-Pending M
'{'	'{'	1	cursor to the first CHAR on the line of the current paragraph	3. Visual Mode
'}'	'}'	1	cursor to the first CHAR on the line of the current paragraph	4. Command-Line Editin
				5. Terminal Mode
(	(	1	cursor N sentences backward	6. Ex Commands
)	)	1	cursor N sentences forward	
star	*	1	search forward for the Nth occurrence of the ident under the cursor	
+	+	1	same as <CR>	
<S-Plus>	<S-+>	1	same as	
CTRL-F				
->	,	1	repeat latest f, t, F or T in opposite direction N times	
=	-	1	cursor to the first CHAR N lines higher	
<S-Minus>	<S-->	1	same as	
CTRL-B				
.	.	2	repeat last change with count replaced with N	

```

// {pattern}<CR> 1 search
forward for the Nth occurrence of {pattern}

//<CR> /<CR> 1 search
forward for {pattern} of last search

0 0 1 cursor to
the first char of the line

count 1 prepend to
command to give a count

count 2 "
count 3 "
count 4 "
count 5 "
count 6 "
count 7 "
count 8 "
count 9 "

: : 1 start
entering an Ex command

N: {count}: start
entering an Ex command with range

from current

line to N-1 lines down

: ; 1 repeat
latest f, t, F or T N times

< <{motion} 2 shift Nmove
lines one 'shiftwidth'
leftwards

<< << 2 shift N
lines one 'shiftwidth' leftwards

= {motion} 2 filter Nmove
lines through "indent"

== == 2 filter N
lines through "indent"

> >{motion} 2 shift Nmove
lines one 'shiftwidth'
rightwards

>> >> 2 shift N
lines one 'shiftwidth' rightwards

? ?{pattern}<CR> 1 search

```

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

backward for the Nth previous occurrence of `{pattern}`

`?<CR>` `?<CR>` 1 search backward for `{pattern}` of last search

`@` `@{a-z}` 2 execute the contents of register `{a-z}`

`@:` `@:` N times repeat the previous ":" command N times

`@@` `@@` 2 repeat the previous `@{a-z}` N times

`A` `A` 2 append text after the end of the line N times

`B` `B` 1 cursor N WORDS backward

`C` `["x]C` 2 change from the cursor position to the end of the line, and N-1 more lines [into register x]; synonym for "c\$"

`D` `["x]D` 2 delete the characters under the cursor until the end of the line and N-1 more lines [into register x]; synonym for "d\$"

`E` `E` 1 cursor forward to the end of WORD N

`F` `F{char}` 1 cursor to the Nth occurrence of `{char}` to the left

`G` `G` 1 cursor to line N, default last line

`H` `H` 1 cursor to line N from top of screen

`I` `I` 2 insert text before the first CHAR on the line N times

[Main](#)
[Commands index](#)
[Quick reference](#)

[1. Insert Mode](#)
[2. Normal Mode](#)
[2.1 Text Objects](#)
[2.2 Window Commands](#)
[2.3 Square Bracket Com](#)
[2.4 Commands Starting](#)
[2.5 Commands Starting](#)
[2.6 Operator-Pending M](#)
[3. Visual Mode](#)
[4. Command-Line Editin](#)
[5. Terminal Mode](#)
[6. Ex Commands](#)

J	J	2	Join N	Main
lines; default is 2				Commands index
K	K		lookup	Quick reference
Keyword under the cursor with				
			'keywordprg'	
L	L	1	cursor to	1. Insert Mode
line N from bottom of screen				2. Normal Mode
M	M	1	cursor to	2.1 Text Objects
middle line of screen				2.2 Window Commands
N	N	1	repeat the	2.3 Square Bracket Com
latest '/' or '?' N times in			opposite	2.4 Commands Starting
direction				2.5 Commands Starting
O	O	2	begin a new	2.6 Operator-Pending M
line above the cursor and			insert text,	3. Visual Mode
repeat N times				4. Command-Line Editin
P	["x]P	2	put the text	5. Terminal Mode
[from register x] before the			cursor N	6. Ex Commands
times				
R	R	2	enter	
replace mode: overtype existing			characters,	
repeat the entered text N-1			times	
S	["x]S	2	delete N	
lines [into register x] and start			insert;	
synonym for "cc".				
I	T{char}	1	cursor till	
after Nth occurrence of {char}			to the left	
U	U	2	undo all	
latest changes on one line			start	
V	V			
linewise Visual mode				
W	W	1	cursor N	
WORDS forward				
X	["x]X	2	delete N	

characters before the cursor [into register x]
`Y` ["x]Y yank N lines
 [into register x]; synonym for `"yy"`
Note: Mapped to `"y$"` by default. [default-mappings](#)
`ZZ` ZZ write if buffer changed and close window
`ZQ` ZQ close window without writing
`[` [{char} square bracket command (see `[` below)
`\` not used
`]`]{char} square bracket command (see `]` below)
`^` ^ 1 cursor to the first CHAR of the line
`-` - 1 cursor to the first CHAR N - 1 lines lower
``` {a-zA-Z0-9} 1 cursor to the mark {a-zA-Z0-9}  
`_(` ( 1 cursor to the start of the current sentence  
`_)` ) 1 cursor to the end of the current sentence  
`<` < 1 cursor to the start of the highlighted area  
`>` > 1 cursor to the end of the highlighted area  
`[` [ 1 cursor to the start of last operated text  
 or start of putted text  
`]` ] 1 cursor to the end of last operated text or end of putted text  
```` `` 1 cursor to the position before latest jump

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

| | | |
|-----------------|---|---|
| <code>{</code> | 1 | cursor to the start of the current paragraph |
| <code>}</code> | 1 | cursor to the end of the current paragraph |
| <code>a</code> | 2 | append text after the cursor N times |
| <code>b</code> | 1 | cursor N words backward |
| <code>c</code> | 2 | delete Nmove text [into register x] and |
| <code>cc</code> | 2 | start insert delete N lines [into register x] and start |
| <code>d</code> | 2 | insert delete Nmove text [into register x] |
| <code>dd</code> | 2 | delete N lines [into register x] |
| <code>do</code> | 2 | same as <code>":diffget"</code> |
| <code>dp</code> | 2 | same as <code>":diffput"</code> |
| <code>e</code> | 1 | cursor forward to the end of word N |
| <code>f</code> | 1 | cursor to Nth occurrence of {char} to the |
| <code>g</code> | | right extended commands, see <code>g</code> below |
| <code>h</code> | 1 | cursor N chars to the left |
| <code>i</code> | 2 | insert text before the cursor N times |
| <code>j</code> | 1 | cursor N lines downward |
| <code>k</code> | 1 | cursor N lines upward |
| <code>l</code> | 1 | cursor N chars to the right |
| <code>m</code> | | set mark {A- |

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

[a-z] at cursor position

n repeat the latest '/' or '?' N times

o begin a new line below the cursor and

repeat N times

p ["x]p
[from register x] after the

times

q q{0-9a-zA-Z"}
characters into named register

(uppercase to append)

q q
recording) stops recording

Q Q
recorded register

q: q:
command-line in command-line window

q/ q/
command-line in command-line window

q? q?
command-line in command-line window

r r{char}
chars with {char}

s ["x]s
delete N characters [into

and start insert

t t{char}
before Nth occurrence of {char}

u u
undo changes

v v
charwise Visual mode

w w
words forward

x ["x]x
delete N

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

characters under and after the

register x]

y ["x]y{motion}

text [into register x]

yy ["x]yy

[into register x]

z z{char}

starting with 'z', see [z](#) below

{ {

paragraphs backward

bar |

column N

} }

paragraphs forward

~ ~

off: switch case of N characters

and move the cursor N

to the right

~ ~{motion}

on: switch case of Nmove text

<C-End> <C-End>

<C-Home> <C-Home>

<C-Left> <C-Left>

<C-LeftMouse> <C-LeftMouse>

keyword at the mouse click

<C-Right> <C-Right>

<C-RightMouse> <C-RightMouse>

"CTRL-T"

<C-Tab> <C-Tab>

"g<Tab>"

 ["x]

N {count}

last digit from {count}

<Down> <Down>

<End> <End>

<F1> <F1>

<Help>

cursor [into

yank Nmove

yank N lines

commands

1 cursor N

1 cursor to

1 cursor N

2 'tildeop'

under cursor

characters

'tildeop'

1 same as "G"

1 same as "gg"

1 same as "b"

":ta" to the

1 same as "w"

same as

same as

2 same as "x"

remove the

1 same as "j"

1 same as "\$"

same as

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

| | | | | |
|---|---|---|--|--|
| <Help> | <Help> | | open a help window | Main |
| <Home> | <Home> | 1 | same as "0" | Commands index |
| <Insert> | <Insert> | 2 | same as "i" | Quick reference |
| <Left> | <Left> | 1 | same as "h" | |
| <LeftMouse> | <LeftMouse> | 1 | move cursor to the mouse click position | 1. Insert Mode |
| <MiddleMouse> | <MiddleMouse> | 2 | same as "gP" at the mouse click position | 2. Normal Mode |
| <PageDown> | <PageDown> | | same as | 2.1 Text Objects |
| CTRL-F | | | | 2.2 Window Commands |
| <PageUp> | <PageUp> | | same as | 2.3 Square Bracket Com |
| CTRL-B | | | | 2.4 Commands Starting |
| <Right> | <Right> | 1 | same as "l" | 2.5 Commands Starting |
| <RightMouse> | <RightMouse> | | start Visual mode, move cursor to the mouse click position | 2.6 Operator-Pending M |
| <S-Down> | <S-Down> | 1 | same as | 3. Visual Mode |
| CTRL-F | | | | 4. Command-Line Editin |
| <S-Left> | <S-Left> | 1 | same as "b" | 5. Terminal Mode |
| <S-LeftMouse> | <S-LeftMouse> | | same as "*" at the mouse click position | 6. Ex Commands |
| <S-Right> | <S-Right> | 1 | same as "w" | |
| <S-RightMouse> | <S-RightMouse> | | same as "#" at the mouse click position | |
| <S-Up> | <S-Up> | 1 | same as | |
| CTRL-B | | | | |
| <Undo> | <Undo> | 2 | same as "u" | |
| <Up> | <Up> | 1 | same as "k" | |
| <ScrollWheelDown> | | | | |
| <ScrollWheelDown> | | | move window three lines down | |
| <S-ScrollWheelDown> | <S-ScrollWheelDown> | | move window one page down | |
| <ScrollWheelUp> | | | | |
| <ScrollWheelUp> | | | move window three lines up | |
| <S-ScrollWheelUp> | <S-ScrollWheelUp> | | | |

`ScrollWheelUp>` move window one page up
`<ScrollWheelLeft>`
`<ScrollWheelLeft>` move window six columns left
`<S-ScrollWheelLeft>` `<S-`
`ScrollWheelLeft>` move window one page left
`<ScrollWheelRight>`
`<ScrollWheelRight>` move window six columns right
`<S-ScrollWheelRight>` `<S-`
`ScrollWheelRight>` move window one page right

2.1 Text objects

objects

These can be used after an operator or in Visual mode to select an object.

| Tag and Visual-mode | Command and action | Op-pending |
|-------------------------|--------------------|---|
| ----- | | |
| v_Quote | a" | double quoted string |
| v_a' | a' | single quoted string |
| v_a(| a(| same as ab |
| v_a) | a) | same as ab |
| v_a< | a< | "a <>" from '<' to the matching '>' |
| v_a> | a> | same as a< |
| v_aB | aB | "a Block" from <code>[{</code> to <code>]}</code> (with brackets) |
| v_aW | aW | "a WORD" (with white space) |
| v_a[| a[| "a []" from '[' to the matching ']' |
| v_a] | a] | same as a[|

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

| | | | |
|----------------------------------|----|--------------|--|
| v_a`</a>             | a` | string in | Main |
| backticks | | | Commands index |
| v_ab | ab | "a block" | Quick reference |
| from "[" to "]" (with braces) | | | |
| v_al | al | all lines | |
| (entire buffer) | | | 1. Insert Mode |
| v_ap | ap | "a | 2. Normal Mode |
| paragraph" (with white space) | | | 2.1 Text Objects |
| v_as | as | "a sentence" | 2.2 Window Commands |
| (with white space) | | | 2.3 Square Bracket Com |
| v_at | at | "a tag | 2.4 Commands Starting |
| block" (with white space) | | | 2.5 Commands Starting |
| v_aw | aw | "a word" | 2.6 Operator-Pending M |
| (with white space) | | | 3. Visual Mode |
| v_a{ | a{ | same as aB | 4. Command-Line Editin |
| v_a} | a} | same as aB | 5. Terminal Mode |
| v_iquote | i" | double | 6. Ex Commands |
| quoted string without the quotes | | | |
| v_i' | i' | single | |
| quoted string without the quotes | | | |
| v_i(| i(| same as ib | |
| v_i) | i) | same as ib | |
| v_i< | i< | "inner <>" | |
| from '<' to the matching '>' | | | |
| v_i> | i> | same as i< | |
| v_iB | iB | "inner | |
| Block" from [{ and]} | | | |
| v_iW | iW | "inner WORD" | |
| v_i[| i[| "inner []" | |
| from '[' to the matching ']' | | | |
| v_i] | i] | same as i[| |
| v_i`</a>             | i` | string in | |
| backticks without the backticks | | | |
| v_ib | ib | "inner | |
| block" from "[" to "]" | | | |
| v_il | il | inner line | |
| (without surrounding whitespace) | | | |
| v_ip | ip | "inner | |
| paragraph" | | | |
| v_is | is | "inner | |

| | | |
|----------------------|----|--------------|
| sentence" | | |
| v_it | it | "inner tag |
| block" | | |
| v_iw | iw | "inner word" |
| v_i{ | i{ | same as iB |
| v_i} | i} | same as iB |

[Main](#)
[Commands index](#)
[Quick reference](#)

2.2 Window commands

CTRL-W

| Tag
action | Command | Normal-mode |
|---------------|---------|-------------|
|---------------|---------|-------------|

| | | |
|-------------------------------|---|---------|
| CTRL-W_CTRL-B | CTRL-W CTRL-B | same as |
|-------------------------------|---|---------|

"CTRL-W b"

| | | |
|-------------------------------|---|-------|
| CTRL-W_CTRL-C | CTRL-W CTRL-C | no-op |
|-------------------------------|---|-------|

| | | |
|-------------------------------|---|---------|
| CTRL-W_CTRL-D | CTRL-W CTRL-D | same as |
|-------------------------------|---|---------|

"CTRL-W d"

| | | |
|-------------------------------|---|---------|
| CTRL-W_CTRL-F | CTRL-W CTRL-F | same as |
|-------------------------------|---|---------|

"CTRL-W f"

| | |
|---|---------|
| CTRL-W CTRL-G | same as |
|---|---------|

"CTRL-W g .."

| | | |
|-------------------------------|---|---------|
| CTRL-W_CTRL-H | CTRL-W CTRL-H | same as |
|-------------------------------|---|---------|

"CTRL-W h"

| | | |
|-------------------------------|---|---------|
| CTRL-W_CTRL-I | CTRL-W CTRL-I | same as |
|-------------------------------|---|---------|

"CTRL-W i"

| | | |
|-------------------------------|---|---------|
| CTRL-W_CTRL-J | CTRL-W CTRL-J | same as |
|-------------------------------|---|---------|

"CTRL-W j"

| | | |
|-------------------------------|---|---------|
| CTRL-W_CTRL-K | CTRL-W CTRL-K | same as |
|-------------------------------|---|---------|

"CTRL-W k"

| | | |
|-------------------------------|---|---------|
| CTRL-W_CTRL-L | CTRL-W CTRL-L | same as |
|-------------------------------|---|---------|

"CTRL-W l"

| | | |
|-------------------------------|---|---------|
| CTRL-W_CTRL-N | CTRL-W CTRL-N | same as |
|-------------------------------|---|---------|

"CTRL-W n"

| | | |
|-------------------------------|---|---------|
| CTRL-W_CTRL-O | CTRL-W CTRL-O | same as |
|-------------------------------|---|---------|

"CTRL-W o"

| | | |
|-------------------------------|---|---------|
| CTRL-W_CTRL-P | CTRL-W CTRL-P | same as |
|-------------------------------|---|---------|

"CTRL-W p"

| | | |
|-------------------------------|---|---------|
| CTRL-W_CTRL-Q | CTRL-W CTRL-Q | same as |
|-------------------------------|---|---------|

[1. Insert Mode](#)
[2. Normal Mode](#)
[2.1 Text Objects](#)
[2.2 Window Commands](#)
[2.3 Square Bracket Com](#)
[2.4 Commands Starting](#)
[2.5 Commands Starting](#)
[2.6 Operator-Pending M](#)
[3. Visual Mode](#)
[4. Command-Line Editin](#)
[5. Terminal Mode](#)
[6. Ex Commands](#)

| | | | |
|---------------------------------|-------------------------------|--------------|--|
| "CTRL-W q" | | | Main |
| CTRL-W_CTRL-R | CTRL-W CTRL-R | same as | Commands index |
| "CTRL-W r" | | | Quick reference |
| CTRL-W_CTRL-S | CTRL-W CTRL-S | same as | |
| "CTRL-W s" | | | |
| CTRL-W_CTRL-T | CTRL-W CTRL-T | same as | 1. Insert Mode |
| "CTRL-W t" | | | 2. Normal Mode |
| CTRL-W_CTRL-V | CTRL-W CTRL-V | same as | 2.1 Text Objects |
| "CTRL-W v" | | | 2.2 Window Commands |
| CTRL-W_CTRL-W | CTRL-W CTRL-W | same as | 2.3 Square Bracket Com |
| "CTRL-W w" | | | 2.4 Commands Starting |
| CTRL-W_CTRL-X | CTRL-W CTRL-X | same as | 2.5 Commands Starting |
| "CTRL-W x" | | | 2.6 Operator-Pending M |
| CTRL-W_CTRL-Z | CTRL-W CTRL-Z | same as | 3. Visual Mode |
| "CTRL-W z" | | | 4. Command-Line Editin |
| CTRL-W_CTRL-] | CTRL-W CTRL-] | same as | 5. Terminal Mode |
| "CTRL-W]" | | | 6. Ex Commands |
| CTRL-W_CTRL-^ | CTRL-W CTRL-^ | same as | |
| "CTRL-W ^" | | | |
| CTRL-W_CTRL-_ | CTRL-W CTRL-_ | same as | |
| "CTRL-W _" | | | |
| CTRL-W_+ | CTRL-W + | increase | |
| current window height N lines | | | |
| CTRL-W_- | CTRL-W - | decrease | |
| current window height N lines | | | |
| CTRL-W_< | CTRL-W < | decrease | |
| current window width N columns | | | |
| CTRL-W_= | CTRL-W = | make all | |
| windows the same height & width | | | |
| CTRL-W_> | CTRL-W > | increase | |
| current window width N columns | | | |
| CTRL-W_H | CTRL-W H | move current | |
| window to the far left | | | |
| CTRL-W_J | CTRL-W J | move current | |
| window to the very bottom | | | |
| CTRL-W_K | CTRL-W K | move current | |
| window to the very top | | | |
| CTRL-W_L | CTRL-W L | move current | |
| window to the far right | | | |
| CTRL-W_P | CTRL-W P | go to | |

| | | | |
|---|---------------------------------|------------------------------|--|
| preview window | | | Main |
| CTRL-W_R | CTRL-W R | rotate | Commands index |
| windows upwards N times | | | Quick reference |
| CTRL-W_S | CTRL-W S | same as | |
| "CTRL-W s" | | | |
| CTRL-W_T | CTRL-W T | move current | 1. Insert Mode |
| window to a new tabpage | | | 2. Normal Mode |
| CTRL-W_W | CTRL-W W | go to N | 2.1 Text Objects |
| previous window (wrap around) | | | 2.2 Window Commands |
| CTRL-W_] | CTRL-W] | split window | 2.3 Square Bracket Com |
| and jump to tag under cursor | | | 2.4 Commands Starting |
| CTRL-W_^ | CTRL-W ^ | split | 2.5 Commands Starting |
| current window and edit alternate | | | 2.6 Operator-Pending M |
| | | file N | 3. Visual Mode |
| CTRL-W__ | CTRL-W _ | set current | 4. Command-Line Editin |
| window height to N (default: | | | 5. Terminal Mode |
| | | very high) | 6. Ex Commands |
| CTRL-W_b | CTRL-W b | go to bottom | |
| window | | | |
| CTRL-W_c | CTRL-W c | close | |
| current window (like :close) | | | |
| CTRL-W_d | CTRL-W d | split window | |
| and jump to definition under | | | |
| | | the cursor | |
| CTRL-W_f | CTRL-W f | split window | |
| and edit file name under the | | | |
| | | cursor | |
| CTRL-W_F | CTRL-W F | split window | |
| and edit file name under the | | | |
| | | cursor and | |
| jump to the line number | | | |
| | | following | |
| the file name. | | | |
| CTRL-W_g_CTRL-] | CTRL-W g CTRL-] | split window | |
| and do :tjump to tag under | | | |
| | | cursor | |
| CTRL-W_g] | CTRL-W g] | split window | |
| and do :tselect for tag | | | |
| | | under cursor | |
| CTRL-W_g} | CTRL-W g } | do a :ptjump | |

to the tag under the cursor
[CTRL-W_gf](#) [CTRL-W g f](#) edit file
 name under the cursor in a new
 tabpage
[CTRL-W_gF](#) [CTRL-W g F](#) edit file
 name under the cursor in a new
 tabpage and
 jump to the line number
 following
 the file name.
[CTRL-W_gt](#) [CTRL-W g t](#) same as [gt](#):
 go to next tabpage
[CTRL-W_gT](#) [CTRL-W g T](#) same as [gT](#):
 go to previous tabpage
[CTRL-W_g<Tab>](#) [CTRL-W g <Tab>](#) same as
[g<Tab>](#): go to last accessed tab
 page
[CTRL-W_h](#) [CTRL-W h](#) go to Nth
 left window (stop at first window)
[CTRL-W_i](#) [CTRL-W i](#) split window
 and jump to declaration of
 identifier
 under the cursor
[CTRL-W_j](#) [CTRL-W j](#) go N windows
 down (stop at last window)
[CTRL-W_k](#) [CTRL-W k](#) go N windows
 up (stop at first window)
[CTRL-W_l](#) [CTRL-W l](#) go to Nth
 right window (stop at last window)
[CTRL-W_n](#) [CTRL-W n](#) open new
 window, N lines high
[CTRL-W_o](#) [CTRL-W o](#) close all
 but current window (like [:only](#))
[CTRL-W_p](#) [CTRL-W p](#) go to
 previous (last accessed) window
[CTRL-W_q](#) [CTRL-W q](#) quit current
 window (like [:quit](#))
[CTRL-W_r](#) [CTRL-W r](#) rotate
 windows downwards N times
[CTRL-W_s](#) [CTRL-W s](#) split

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

current window in two parts, new window N

lines high

[CTRL-W t](#) CTRL-W t go to top

window

[CTRL-W v](#) CTRL-W v split

current window vertically, new window N columns

wide

[CTRL-W w](#) CTRL-W w go to N next

window (wrap around)

[CTRL-W x](#) CTRL-W x exchange

current window with window N (default: next window)

[CTRL-W z](#) CTRL-W z close

preview window

[CTRL-W |](#) CTRL-W | set window

width to N columns

[CTRL-W }](#) CTRL-W } show tag

under cursor in preview window

[CTRL-W <Down>](#) CTRL-W <Down> same as

"CTRL-W j"

[CTRL-W <Up>](#) CTRL-W <Up> same as

"CTRL-W k"

[CTRL-W <Left>](#) CTRL-W <Left> same as

"CTRL-W h"

[CTRL-W <Right>](#) CTRL-W <Right> same as

"CTRL-W l"

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

2.3 Square bracket commands []

| Tag | Char | Note | Normal-mode |
|------------------------------------|----------|---------|-------------|
| ----- | | | |
| [_CTRL-D | [CTRL-D | jump to | |
| first #define found in current and | | | |
| included | | | |

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

files matching the word under the
start searching at beginning of

[\[_CTRL-I](#) [CTRL-I
first line in current and included

contains the word under the
start searching at beginning of

[\# [# 1
previous unmatched #if, #else

[\['](#) [' 1
previous lowercase mark, on first

[\[\(](#) [(1
times back to unmatched '('

[\[star](#) [* 1 same as "[/"

[\[`](#)                      [` 1 cursor to
previous lowercase mark

[\[/](#) [/ 1 cursor to N
previous start of a C comment

[\[D](#) [D 1 list all
defines found in current and

files matching the word under the
start searching at beginning of

[\[I](#) [I 1
lines found in current and

files that contain the word under
start searching at beginning of

[\[P](#) [P 2 same as "[p"
[\[\[](#) [[1 cursor N

cursor,
current file
jump to

files that

cursor,
current file
cursor to N

or #ifdef
cursor to

non-blank
cursor N

same as "[/"
cursor to

cursor to N

list all

included

cursor,

current file
list all

included

the cursor,

current file
same as "[p"
cursor N

sections backward
`[]` `[]` 1 cursor N
 SECTIONS backward
`[c` `[c` 1 cursor N
 times backwards to start of change
`[d` `[d` show first
 #define found in current and
 included
 files matching the word under the
 cursor,
 start searching at beginning of
 current file
`[f` `[f` same as "gf"
`[i` `[i` show first
 line found in current and
 included
 files that contains the word under
 the cursor,
 start searching at beginning of
 current file
`[m` `[m` 1 cursor N
 times back to start of member
 function
`[p` `[p` 2 like "P",
 but adjust indent to current line
`[s` `[s` 1 move to the
 previous misspelled word
`[z` `[z` 1 move to
 start of open fold
`[{` `[{` 1 cursor N
 times back to unmatched '{'
`[<MiddleMouse>` `[<MiddleMouse>` 2 same as "[p"

`]_CTRL-D` `] CTRL-D` jump to
 first #define found in current and
 included
 files matching the word under the
 cursor,
 start searching at cursor position
`]_CTRL-I` `] CTRL-I` jump to

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

first line in current and included files that contains the word under the cursor, start searching at cursor position

[\]#](#) [\]#](#) 1 cursor to N next unmatched #endif or #else

[\]'](#) [\]'](#) 1 cursor to next lowercase mark, on first non-blank cursor N

[\]\)](#) [\]\)](#) 1 cursor N times forward to unmatched ')'

[\]star](#) [\]*](#) 1 same as "]/"

[\]`](#) [\]`](#) 1 cursor to next lowercase mark

[\]/](#) [\]/](#) 1 cursor to N next end of a C comment

[\]D](#) [\]D](#) list all #defines found in current and included files matching the word under the cursor, start searching at cursor position

[\]I](#) [\]I](#) list all lines found in current and included files that contain the word under the cursor, start searching at cursor position

[\]P](#) [\]P](#) 2 same as "[p"

[\]\[](#) [\]\[](#) 1 cursor N SECTIONS forward

[\]\]](#) [\]\]](#) 1 cursor N sections forward

[\]c](#) [\]c](#) 1 cursor N times forward to start of change

[\]d](#) [\]d](#) show first #define found in current and included

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

files matching the word under the cursor,
start searching at cursor position
`]f` same as "`gf`"
`]i` show first
line found in current and included
files that contains the word under the cursor,
start searching at cursor position
`]m` 1 cursor N
times forward to end of member
`]p` 2 function
but adjust indent to current line like "`p`",
`]s` 1 move to next
misspelled word
`]z` 1 move to end
of open fold
`]}` 1 cursor N
times forward to unmatched '`}`'
`<MiddleMouse>` 2 same as "`]p`"

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

2.4 Commands starting with '`g`' `g`

| Tag | Char | Note | Normal-mode action |
|-----------------------|------------------------------------|----------------------------|---|
| <code>g_CTRL-G</code> | <code>g</code> <code>CTRL-G</code> | show | information about current cursor position |
| <code>g_CTRL-H</code> | <code>g</code> <code>CTRL-H</code> | start Select | block mode |
| <code>g_CTRL-]</code> | <code>g</code> <code>CTRL-]</code> | <code>:tjump</code> to | the tag under the cursor |
| <code>g#</code> | <code>g#</code> | 1 like " <code>#</code> ", | but without using " <code>\<</code> " and " <code>\></code> " |

| | | | |
|----------------------------------|--------------------------|---|---|
| g\$ | g\$ | 1 | when 'wrap' |
| off go to rightmost character of | | | the current |
| line that is on the screen; | | | |
| on go to the rightmost character | | | when 'wrap' |
| | | | of the |
| current screen line | | | |
| g& | g& | 2 | repeat last |
| ":s" on all lines | | | |
| g' | g'{mark} | 1 | like ' but |
| without changing the jumplist | | | |
| g`</a>               | <a href="#">g`{mark} | 1 | like ` but |
| without changing the jumplist | | | |
| gstar | g* | 1 | like " " ,
but without using "\<" and "\>" |
| g± | g± | | go to newer |
| text state N times | | | |
| g, | g, | 1 | go to N |
| newer position in change list | | | |
| g- | g- | | go to older |
| text state N times | | | |
| g@ | g@ | 1 | when 'wrap' |
| off go to leftmost character of | | | the current |
| line that is on the screen; | | | |
| on go to the leftmost character | | | when 'wrap' |
| | | | of the |
| current screen line | | | |
| g8 | g8 | | print hex |
| value of bytes used in UTF-8 | | | |
| | | | character |
| under the cursor | | | |
| g; | g; | 1 | go to N |
| older position in change list | | | |
| g< | g< | | display |
| previous command output | | | |
| g? | g? | 2 | Rot13 |
| encoding operator | | | |

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

| | | | | |
|---------------------------------------|----------------------------|-----|--------------|--|
| g?g? | g?? | 2 | Rot13 encode | Main |
| current line | | | | Commands index |
| g?g? | g?g? | 2 | Rot13 encode | Quick reference |
| current line | | | | |
| gD | gD | 1 | go to | 1. Insert Mode |
| definition of word under the cursor | | | in current | 2. Normal Mode |
| file | | | | 2.1 Text Objects |
| gE | gE | 1 | go backwards | 2.2 Window Commands |
| to the end of the previous | | | WORD | 2.3 Square Bracket Com |
| gH | gH | | start Select | 2.4 Commands Starting |
| line mode | | | | 2.5 Commands Starting |
| gI | gI | 2 | like "I", | 2.6 Operator-Pending M |
| but always start in column 1 | | | | 3. Visual Mode |
| gJ | gJ | 2 | join lines | 4. Command-Line Editin |
| without inserting space | | | | 5. Terminal Mode |
| gN | gN | 1,2 | find the | 6. Ex Commands |
| previous match with the last used | | | search | |
| pattern and Visually select it | | | | |
| gP | ["x]gP | 2 | put the text | |
| [from register x] before the | | | cursor N | |
| times, leave the cursor after it | | | | |
| gQ | gQ | | switch to | |
| "Ex" mode with Vim editing | | | | |
| gR | gR | 2 | enter | |
| Virtual Replace mode | | | | |
| gT | gT | | go to the | |
| previous tabpage | | | | |
| gU | gU{motion} | 2 | make Nmove | |
| text uppercase | | | | |
| gV | gV | | don't | |
| reselect the previous Visual area | | | when | |
| executing a mapping or menu in Select | | | mode | |
| g] | g] | | :tselect on | |
| the tag under the cursor | | | | |

[g^](#) [g^](#) 1 when ['wrap'](#)
off go to leftmost non-white
character of
the current line that is on
the screen;
when ['wrap'](#) on go to the
leftmost
non-white character of the current
screen line
cursor to
[g_](#) [g_](#) 1 the last CHAR N - 1 lines lower
[ga](#) [ga](#) print ascii
value of character under the
cursor
[gd](#) [gd](#) 1 go to
definition of word under the cursor
in current
function
[ge](#) [ge](#) 1 go backwards
to the end of the previous
word
[gf](#) [gf](#) start
editing the file whose name is under
the cursor
[gE](#) [gE](#) start
editing the file whose name is under
the cursor
and jump to the line number
following
the filename.
[gg](#) [gg](#) 1 cursor to
line N, default first line
[gh](#) [gh](#) start Select
mode
[gi](#) [gi](#) 2 like "i",
but first move to the ['^](#) mark
[gj](#) [gj](#) 1 like "j",
but when ['wrap'](#) on go N screen
lines down
[ak](#) [ak](#) 1 like "k".

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

but when ['wrap'](#) on go N screen

[gm](#) [gm](#) 1 go to lines up
character at middle of the screenline

[gM](#) [gM](#) 1 go to
character at middle of the text line

[gn](#) [gn](#) 1,2 find the
next match with the last used
search

pattern and Visually select it

[go](#) [go](#) 1 cursor to
byte N in the buffer

[gp](#) [\["x\]gp](#) 2 put the text
[from register x] after the
cursor N

times, leave the cursor after it

[gq](#) [gq{motion}](#) 2 format Nmove
text

[gr](#) [gr{char}](#) 2 virtual
replace N chars with [{char}](#)

[gs](#) [gs](#) go to sleep
for N seconds (default 1)

[gt](#) [gt](#) go to the
next tabpage

[gu](#) [gu{motion}](#) 2 make Nmove
text lowercase

[gv](#) [gv](#) reselect the
previous Visual area

[gw](#) [gw{motion}](#) 2 format Nmove
text and keep cursor

[gx](#) [gx](#) execute
application for filepath at cursor

[g@](#) [g@{motion}](#) call
['operatorfunc'](#)

[g~](#) [g~{motion}](#) 2 swap case
for Nmove text

[g<Down>](#) [g<Down>](#) 1 same as "gj"

[g<End>](#) [g<End>](#) 1 same as "g\$"

but go to the rightmost
non-blank

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

character instead

| | | | |
|-------------------------------------|-----------------------------------|---|-----------------------------|
| g<Home> | <code>g<Home></code> | 1 | same as "g0" |
| g<LeftMouse> | <code>g<LeftMouse></code> | | same as <code><C-</code> |
| LeftMouse> | | | |
| | <code>g<MiddleMouse></code> | | same as <code><C-</code> |
| MiddleMouse> | | | |
| g<RightMouse> | <code>g<RightMouse></code> | | same as <code><C-</code> |
| RightMouse> | | | |
| g<Tab> | <code>g<Tab></code> | | go to last |
| accessed tabpage | | | |
| g<Up> | <code>g<Up></code> | 1 | same as "gk" |

2.5 Commands starting with 'z' z

| Tag | Char | Note | Normal-mode |
|----------------------------------|----------------------------------|--------------|-------------|
| action | | | |
| ----- | | | |
| z<CR> | <code>z<CR></code> | redraw, | |
| cursor line to top of window, | | | |
| | | cursor on | |
| first non-blank | | | |
| zN<CR> | <code>z{height}<CR></code> | redraw, make | |
| window {height} lines high | | | |
| z+ | <code>z+</code> | cursor on | |
| line N (default line below | | | |
| | | window), | |
| otherwise like "z<CR>" | | | |
| z- | <code>z-</code> | redraw, | |
| cursor line at bottom of window, | | | |
| | | cursor on | |
| first non-blank | | | |
| z. | <code>z.</code> | redraw, | |
| cursor line to center of window, | | | |
| | | cursor on | |
| first non-blank | | | |
| z= | <code>z=</code> | give | |
| spelling suggestions | | | |
| z^ | <code>z^</code> | open a | |

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

| | | | |
|---|--------------------|-----------------------------|--|
| zA | zA | open a | Main |
| closed fold or close an open fold | | | Commands index |
| | | recursively | Quick reference |
| zC | zC | close folds | |
| recursively | | | |
| zD | zD | delete folds | 1. Insert Mode |
| recursively | | | 2. Normal Mode |
| zE | zE | eliminate | 2.1 Text Objects |
| all folds | | | 2.2 Window Commands |
| zF | zF | create a | 2.3 Square Bracket Com |
| fold for N lines | | | 2.4 Commands Starting |
| zG | zG | temporarily | 2.5 Commands Starting |
| mark word as correctly spelled | | | 2.6 Operator-Pending M |
| zH | zH | when 'wrap' | 3. Visual Mode |
| off scroll half a screenwidth | | | 4. Command-Line Editin |
| | | to the right | 5. Terminal Mode |
| zL | zL | when 'wrap' | 6. Ex Commands |
| off scroll half a screenwidth | | | |
| | | to the left | |
| zM | zM | set | |
| 'foldlevel' to zero | | | |
| zN | zN | set | |
| 'foldenable' | | | |
| zO | zO | open folds | |
| recursively | | | |
| zR | zR | set | |
| 'foldlevel' to the deepest fold | | | |
| zW | zW | temporarily | |
| mark word as incorrectly spelled | | | |
| zX | zX | re-apply | |
| 'foldlevel' | | | |
| z^ | z^ | cursor on | |
| line N (default line above | | | |
| | | window), | |
| otherwise like "z-" | | | |
| za | za | open a | |
| closed fold, close an open fold | | | |
| zb | zb | redraw, | |
| cursor line at bottom of window | | | |
| zc | zc | close a fold | |
| zd | zd | delete a | |

| | | | |
|------------------------------------|----------------------------|-----------------------------|--|
| zu | zu | delete a | Main |
| fold | | | Commands index |
| ze | ze | when 'wrap' | Quick reference |
| off scroll horizontally to | | position the | |
| cursor at the end (right side) | | of the | |
| screen | | create a | 1. Insert Mode |
| zf | zf{motion} | permanently | 2. Normal Mode |
| fold for Nmove text | | when 'wrap' | 2.1 Text Objects |
| zg | zg | to the right | 2.2 Window Commands |
| mark word as correctly spelled | | toggle | 2.3 Square Bracket Com |
| zh | zh | 1 move to the | 2.4 Commands Starting |
| off scroll screen N characters | | 1 move to the | 2.5 Commands Starting |
| zi | zi | when 'wrap' | 2.6 Operator-Pending M |
| 'foldenable' | | to the left | 3. Visual Mode |
| zj | zj | subtract one | 4. Command-Line Editin |
| start of the next fold | | reset | 5. Terminal Mode |
| zk | zk | open fold | 6. Ex Commands |
| end of the previous fold | | paste in | |
| zl | zl | paste in | |
| off scroll screen N characters | | add one to | |
| zm | zm | when 'wrap' | |
| from 'foldlevel' | | position the | |
| zn | zn | side) of the | |
| 'foldenable' | | | |
| zo | zo | | |
| zp | zp | | |
| block-mode without trailing spaces | | | |
| zP | zP | | |
| block-mode without trailing spaces | | | |
| zr | zr | | |
| 'foldlevel' | | | |
| zs | zs | | |
| off scroll horizontally to | | | |
| cursor at the start (left | | | |
| screen | | | |

Screen

| | | |
|----------------------------------|----------|-------------------------|
| zt | zt | redraw, |
| cursor line at top of window | | |
| zuw | zuw | undo zw |
| zug | zug | undo zg |
| zuW | zuW | undo zW |
| zuG | zuG | undo zG |
| zv | zv | open enough |
| folds to view the cursor line | | |
| zw | zw | permanently |
| mark word as incorrectly spelled | | |
| zx | zx | re-apply |
| 'foldlevel' and do "zv" | | |
| zy | zy | yank without |
| trailing spaces | | |
| zz | zz | redraw, |
| cursor line at center of window | | |
| z<Left> | z<Left> | same as "zh" |
| z<Right> | z<Right> | same as "zl" |

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

2.6 Operator-pending mode

[operator-pending-index](#)

These can be used after an operator, but before a `{motion}` has been entered.

| Tag | Char | Operator- |
|--------------------------|------------------------|----------------|
| pending-mode | action | |
| ----- | | |
| o_v | v | force operator |
| to work charwise | | |
| o_V | V | force operator |
| to work linewise | | |
| o_CTRL-V | CTRL-V | force operator |
| to work blockwise | | |

3. Visual mode

[visual-index](#)

Most commands in Visual mode are the same as in Normal mode. The ones listed here are those that are different.

| Tag | Command | Note | Visual-mode action |
|---------------------------------|---|------|--|
| ----- | | | |
| v_CTRL-_CTRL-N | CTRL-\ CTRL-N | | stop Visual mode |
| v_CTRL-_CTRL-G | CTRL-\ CTRL-G | | go to Normal mode |
| v_CTRL-A | CTRL-A | 2 | add N to number in highlighted text |
| v_CTRL-C | CTRL-C | | stop Visual mode |
| v_CTRL-G | CTRL-G | | toggle between Visual mode and Select mode |
| v_<BS> | <BS> | 2 | Select mode: delete highlighted area |
| v_CTRL-H | CTRL-H | 2 | same as <BS> |
| v_CTRL-O | CTRL-O | | switch from Select to Visual mode for one command |
| v_CTRL-V | CTRL-V | | make Visual mode blockwise or stop Visual mode |
| v_CTRL-X | CTRL-X | 2 | subtract N from number in highlighted text |
| v_<Esc> | <Esc> | | stop Visual mode |
| v_CTRL-] | CTRL-] | | jump to highlighted tag |
| v_! | !{filter} | 2 | filter the highlighted lines through the external command {filter} |
| v_: | : | | start a command-line with the highlighted |

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

| | | | | |
|---|---|---|------------------------------|--|
| | | | lines as a | Main |
| range | | | | Commands index |
| v_< | < | 2 | shift the | Quick reference |
| highlighted lines one | | | | |
| | | | 'shiftwidth' | |
| left | | | | 1. Insert Mode |
| v_= | = | 2 | filter the | 2. Normal Mode |
| highlighted lines through the | | | | 2.1 Text Objects |
| | | | external | 2.2 Window Commands |
| program given with the 'equalprg' | | | option | 2.3 Square Bracket Com |
| v_> | > | 2 | shift the | 2.4 Commands Starting |
| highlighted lines one | | | | 2.5 Commands Starting |
| | | | 'shiftwidth' | 2.6 Operator-Pending M |
| right | | | | 3. Visual Mode |
| v_b_A | A | 2 | block mode: | 4. Command-Line Editin |
| append same text in all lines, | | | after the | 5. Terminal Mode |
| | | | | 6. Ex Commands |
| highlighted area | | | | |
| v_C | C | 2 | delete the | |
| highlighted lines and start | | | | |
| | | | insert | |
| v_D | D | 2 | delete the | |
| highlighted lines | | | | |
| v_b_I | I | 2 | block mode: | |
| insert same text in all lines, | | | before the | |
| | | | | |
| highlighted area | | | | |
| v_J | J | 2 | join the | |
| highlighted lines | | | | |
| v_K | K | | run | |
| 'keywordprg' | | | on the highlighted area | |
| v_O | O | | move | |
| horizontally to other corner of area | | | | |
| v_P | P | | replace | |
| highlighted area with register | | | | |
| | | | contents; | |
| registers are unchanged | | | | |
| v_R | R | 2 | delete the | |
| highlighted lines and start | | | | |

| | | | |
|-------------------------|----|---|--|
| v_S | S | 2 | insert
delete the
highlighted lines and start |
| v_U | U | 2 | insert
make
highlighted area uppercase |
| v_V | V | | make Visual
mode linewise or stop Visual |
| v_X | X | 2 | mode
delete the
highlighted lines |
| v_Y | Y | | yank the
highlighted lines |
| v_Quote | " | | extend
quoted
highlighted area with a double
string |
| v_a' | ' | | extend
quoted
highlighted area with a single
string |
| v_a(| (| | same as ab |
| v_a) |) | | same as ab |
| v_a< | < | | extend
highlighted area with a <> block |
| v_a> | > | | same as a< |
| v_aB | B | | extend
highlighted area with a {} block |
| v_aW | W | | extend
highlighted area with "a WORD" |
| v_a[| [| | extend
highlighted area with a [] block |
| v_a] |] | | same as a[|
| v_a`</a>    | ` | | extend
highlighted area with a backtick
string |
| v_ab | ab | | extend
highlighted area with a () block |
| v_al | al | | extend |

[Main](#)
[Commands index](#)
[Quick reference](#)

[1. Insert Mode](#)
[2. Normal Mode](#)
[2.1 Text Objects](#)
[2.2 Window Commands](#)
[2.3 Square Bracket Com](#)
[2.4 Commands Starting](#)
[2.5 Commands Starting](#)
[2.6 Operator-Pending M](#)
[3. Visual Mode](#)
[4. Command-Line Editin](#)
[5. Terminal Mode](#)
[6. Ex Commands](#)

| | | | |
|--------------------------------------|----------|---|------------|
| highlighted area to all lines | | | |
| v_ap | ap | | extend |
| highlighted area with a paragraph | | | |
| v_as | as | | extend |
| highlighted area with a sentence | | | |
| v_at | at | | extend |
| highlighted area with a tag block | | | |
| v_aw | aw | | extend |
| highlighted area with "a word" | | | |
| v_a{ | a{ | | same as aB |
| v_a} | a} | | same as aB |
| v_c | c | 2 | delete |
| highlighted area and start insert | | | |
| v_d | d | 2 | delete |
| highlighted area | | | |
| v_g_CTRL-A | g CTRL-A | 2 | add N to |
| number in highlighted text | | | |
| v_g_CTRL-X | g CTRL-X | 2 | subtract N |
| from number in highlighted text | | | |
| v_gJ | gJ | 2 | join the |
| highlighted lines without | | | inserting |
| spaces | | | |
| v_gq | gq | 2 | format the |
| highlighted lines | | | |
| v_gv | gv | | exchange |
| current and previous highlighted | | | area |
| v_iquote | i" | | extend |
| highlighted area with a double | | | quoted |
| string (without quotes) | | | |
| v_i' | i' | | extend |
| highlighted area with a single | | | quoted |
| string (without quotes) | | | |
| v_i(| i(| | same as ib |
| v_i) | i) | | same as ib |
| v_i< | i< | | extend |
| highlighted area with inner <> block | | | |

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

| | | |
|---------------------------------------|----|--------------|
| v_i> | i> | same as i< |
| v_iB | iB | extend |
| highlighted area with inner {} block | | |
| v_iW | iW | extend |
| highlighted area with "inner WORD" | | |
| v_i[| i[| extend |
| highlighted area with inner [] block | | |
| v_i] | i] | same as i[|
| v_i`</a>                  | i` | extend |
| highlighted area with a backtick | | |
| | | quoted |
| string (without the backticks) | | |
| v_ib | ib | extend |
| highlighted area with inner () block | | |
| v_il | il | select inner |
| line under cursor | | |
| v_ip | ip | extend |
| highlighted area with inner paragraph | | |
| v_is | is | extend |
| highlighted area with inner sentence | | |
| v_it | it | extend |
| highlighted area with inner tag block | | |
| v_iw | iw | extend |
| highlighted area with "inner word" | | |
| v_i{ | i{ | same as iB |
| v_i} | i} | same as iB |
| v_o | o | move cursor |
| to other corner of area | | |
| v_p | p | replace |
| highlighted area with register | | |
| | | contents; |
| deleted text in unnamed register | | |
| v_r | r | 2 replace |
| highlighted area with a character | | |
| v_s | s | 2 delete |
| highlighted area and start insert | | |
| v_u | u | 2 make |
| highlighted area lowercase | | |
| v_v | v | make Visual |
| mode charwise or stop | | |

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

| | | | |
|---------------------|---|---|---|
| v_x | x | 2 | Visual mode delete the highlighted area |
| v_y | y | | yank the highlighted area |
| v_~ | ~ | 2 | swap case for the highlighted area |

[Main](#)
[Commands index](#)
[Quick reference](#)

[1. Insert Mode](#)
[2. Normal Mode](#)
[2.1 Text Objects](#)
[2.2 Window Commands](#)
[2.3 Square Bracket Com](#)
[2.4 Commands Starting](#)
[2.5 Commands Starting](#)
[2.6 Operator-Pending M](#)
[3. Visual Mode](#)
[4. Command-Line Editin](#)
[5. Terminal Mode](#)
[6. Ex Commands](#)

4. Command-line editing [ex-edit-index](#)

Get to the command-line with the ':', '!', '/' or '?' commands.

Normal characters are inserted at the current cursor position.

"Completion" below refers to context-sensitive completion. It will complete file names, tags, commands etc. as appropriate.

| Tag | Command | Command-line |
|--------------------------|------------------------|--|
| editing-mode | action | |
| ----- | | |
| | CTRL-@ | not used |
| c_CTRL-A | CTRL-A | do completion on the pattern in front of the cursor and insert all matches |
| c_CTRL-B | CTRL-B | cursor to begin of command-line |
| c_CTRL-C | CTRL-C | same as <Esc> |
| c_CTRL-D | CTRL-D | list completions that match the pattern in front of the cursor |
| c_CTRL-E | CTRL-E | cursor to end of command-line |
| 'cedit' | CTRL-F | default value for 'cedit' : opens the command-line |

window; otherwise not used

[c_CTRL-G](#) CTRL-G next match when
'[incsearch](#)' is active

[c_<BS>](#) <BS> delete the
character in front of the cursor

[c_digraph](#) {char1} <BS> {char2} enter digraph

when '[digraph](#)' is on

[c_CTRL-H](#) CTRL-H same as <BS>

[c_<Tab>](#) <Tab> if '[wildchar](#)'
is <Tab>: Do completion on
the pattern in
front of the cursor

[c_<S-Tab>](#) <S-Tab> same as CTRL-P

[c_wildchar](#) '[wildchar](#)' Do completion
on the pattern in front of the
cursor
(default: <Tab>)

[c_CTRL-I](#) CTRL-I same as <Tab>

[c_<NL>](#) <NL> same as <CR>

[c_CTRL-J](#) CTRL-J same as <CR>

[c_CTRL-K](#) CTRL-K {char1} {char2} enter digraph

[c_CTRL-L](#) CTRL-L do completion
on the pattern in front of the
cursor and
insert the longest common part

[c_<CR>](#) <CR> execute entered
command

[c_CTRL-M](#) CTRL-M same as <CR>

[c_CTRL-N](#) CTRL-N after using
'[wildchar](#)' with multiple matches:
go to next
match, otherwise: recall older
command-line
from history.

CTRL-O not used

[c_CTRL-P](#) CTRL-P after using
'[wildchar](#)' with multiple matches:
go to previous

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

match, otherwise: recall older
command-line

from history.
[c_CTRL-Q](#) [CTRL-Q](#) same as [CTRL-V](#),
unless it's used for terminal

[c_CTRL-R](#) [CTRL-R {regname}](#) control flow
insert the
contents of a register or object
under the
cursor as if typed
[c_CTRL-R_CTRL-R](#) [CTRL-R CTRL-R {regname}](#)
[c_CTRL-R_CTRL-O](#) [CTRL-R CTRL-O {regname}](#)
insert the
contents of a register or object
under the
cursor literally

[CTRL-S](#) not used, or
used for terminal control flow

[c_CTRL-T](#) [CTRL-T](#) previous match
when '[incsearch](#)' is active

[c_CTRL-U](#) [CTRL-U](#) remove all
characters between the cursor
and beginning
of command-line

[c_CTRL-V](#) [CTRL-V](#) insert next
non-digit literally, insert three
digit decimal
number as a single byte.

[c_CTRL-W](#) [CTRL-W](#) delete the word
in front of the cursor

[CTRL-X](#) not used
(reserved for completion)

[CTRL-Y](#) copy (yank)
modeless selection

[CTRL-Z](#) not used
(reserved for suspend)

[c_<Esc>](#) [<Esc>](#) abandon
command-line without executing it

[c_CTRL-\[](#) [CTRL-\[](#) same as [<Esc>](#)

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

| | | | |
|-----------------------------------|---------------------------------|------------------------|---|
| c_CTRL-_CTRL-N | CTRL-\ | CTRL-N | go to Normal mode, abandon command-line |
| c_CTRL-_CTRL-G | CTRL-\ | CTRL-G | go to Normal mode, abandon command-line |
| | CTRL-\ | a - d | reserved for extensions |
| c_CTRL-_e | CTRL-\ | e {expr} | replace the command line with the result of {expr} |
| | CTRL-\ | f - z | reserved for extensions |
| | CTRL-\ | others | not used |
| c_CTRL-] | CTRL-] | | trigger abbreviation |
| c_CTRL-^ | CTRL-^ | | toggle use of :lmap mappings |
| c_ | | | delete the character under the cursor |
| c_<Left> | <Left> | | cursor left |
| c_<S-Left> | <S-Left> | | cursor one word left |
| c_<C-Left> | <C-Left> | | cursor one word left |
| c_<Right> | <Right> | | cursor right |
| c_<S-Right> | <S-Right> | | cursor one word right |
| c_<C-Right> | <C-Right> | | cursor one word right |
| c_<Up> | <Up> | | recall previous command-line from history that matches pattern in front of the cursor |
| c_<S-Up> | <S-Up> | | recall previous command-line from history |
| c_<Down> | <Down> | | recall next command-line from history that matches pattern in front of the cursor |
| c_<S-Down> | <S-Down> | | recall next |

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

command-line from history
[c_<Home>](#) [<Home>](#) cursor to start
 of command-line
[c_<End>](#) [<End>](#) cursor to end
 of command-line
[c_<PageDown>](#) [<PageDown>](#) same as [<S-Down>](#)
[c_<PageUp>](#) [<PageUp>](#) same as [<S-Up>](#)
[c_<Insert>](#) [<Insert>](#) toggle insert/
 overstrike mode
[c_<LeftMouse>](#) [<LeftMouse>](#) cursor at mouse
 click

commands in wildmenu mode (see ['wildmenu'](#))

[<Up>](#) move up to
 parent
[<Down>](#) move down to
 submenu
[<Left>](#) select the
 previous match
[<Right>](#) select the next
 match
[<CR>](#) move into
 submenu when doing menu completion
[CTRL-E](#) stop completion
 and go back to original text
[CTRL-Y](#) accept selected
 match and stop completion
 other stop completion
 and insert the typed character

commands in wildmenu mode with ['wildoptions'](#)
 set to "pum"

[<PageUp>](#) select a match
 several entries back
[<PageDown>](#) select a match
 several entries forward

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

5. Terminal mode [terminal-mode-index](#)

In a [terminal](#) buffer all keys except `CTRL-\` are forwarded to the terminal job. If `CTRL-\` is pressed, the next key is forwarded unless it is `CTRL-N` or `CTRL-O`.

Use `CTRL-\_CTRL-N` to go to Normal mode.

Use `t_CTRL-\_CTRL-O` to execute one normal mode command and then return to terminal mode.

You found it,
Arthur!

holy-grail

6. EX commands

[Ex-command](#) [ex-cmd-index](#) [:index](#)

This is a brief but complete listing of all the ":" commands, without mentioning any arguments. The optional part of the command name is inside [].
The commands are sorted on the non-optional part of their name.

| Tag | Command | Action |
|--|----------|-----------------|
| <hr/> | | |
| : | : | nothing |
| :range
in {range} | :{range} | go to last line |
| :!
execute an external command | :! | filter lines or |
| :!!
":!" command | :!! | repeat last |
| : #
":number" | :# | same as |
| :&
.. | :& | repeat last |

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

| | | | |
|------------------------------------|---------------|-----------------|--|
| ":substitute" | | | Main |
| :star | :* | use the last | Commands index |
| Visual area, like ":'<,'>" | | | Quick reference |
| :< | :< | shift lines one | |
| 'shiftwidth' left | | | |
| := | := | print the last | 1. Insert Mode |
| line number | | | 2. Normal Mode |
| :> | :> | shift lines one | 2.1 Text Objects |
| 'shiftwidth' right | | | 2.2 Window Commands |
| :@ | :@ | execute | 2.3 Square Bracket Com |
| contents of a register | | | 2.4 Commands Starting |
| :@@ | :@@ | repeat the | 2.5 Commands Starting |
| previous ":@" | | | 2.6 Operator-Pending M |
| :2match | :2mat[ch] | define a second | 3. Visual Mode |
| match to highlight | | | 4. Command-Line Editin |
| :3match | :3mat[ch] | define a third | 5. Terminal Mode |
| match to highlight | | | 6. Ex Commands |
| :Next | :N[ext] | go to previous | |
| file in the argument list | | | |
| :append | :a[ppend] | append text | |
| :abbreviate | :ab[breviate] | enter | |
| abbreviation | | | |
| :abclear | :abc[lear] | remove all | |
| abbreviations | | | |
| :aboveleft | :abo[veleft] | make split | |
| window appear left or above | | | |
| :all | :al[l] | open a window | |
| for each file in the argument | | | |
| | | | list |
| :amenu | :am[enu] | enter new menu | |
| item for all modes | | | |
| :anoremenu | :an[oremenu] | enter a new | |
| menu for all modes that will not | | | |
| | | | be remapped |
| :args | :ar[gs] | print the | |
| argument list | | | |
| :argadd | :arga[dd] | add items to | |
| the argument list | | | |
| :argdedupe | :argded[upe] | remove | |
| duplicates from the argument list | | | |

| | | |
|-----------------------------|---------------|--|
| :argdelete | :argd[elete] | delete items from the argument list |
| :argedit | :arge[dit] | add item to the argument list and edit it |
| :argdo | :argdo | do a command on all items in the argument list |
| :argglobal | :argg[lobal] | define the global argument list |
| :arglocal | :argl[ocal] | define a local argument list |
| :argument | :argu[ment] | go to specific file in the argument list |
| :ascii | :as[cii] | print ascii value of character under the cursor |
| :autocmd | :au[tocmd] | enter or show autocommands |
| :augroup | :aug[roup] | select the autocommand group to use |
| :aunmenu | :aun[menu] | remove menu for all modes |
| :buffer | :b[uffer] | go to specific buffer in the buffer list |
| :bNext | :bN[ext] | go to previous buffer in the buffer list |
| :ball | :ba[ll] | open a window for each buffer in the buffer list |
| :badd | :bad[d] | add buffer to the buffer list |
| :balt | :balt | like ":badd" but also set the alternate file |
| :bdelete | :bd[elete] | remove a buffer from the buffer list |
| :belowright | :bel[owright] | make split window appear right or below |
| :bfirst | :bf[irst] | go to first buffer in the buffer list |
| :blast | :bl[ast] | go to last buffer in the buffer list |
| :bmodified | :bm[odified] | go to next buffer in the buffer list that has |

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

| | | |
|-----------------------------|-------------------------------|--|
| :bnext | :bn[ext] | been modified
go to next |
| buffer in the buffer list | | |
| :botright | :bo[tright] | make split
window appear at bottom or far right |
| :bprevious | :bp[revious] | go to previous
buffer in the buffer list |
| :brewind | :br[ewind] | go to first
buffer in the buffer list |
| :break | :brea[k] | break out of
while loop |
| :breakadd | :breaka[dd] | add a debugger
breakpoint |
| :breakdel | :breakd[el] | delete a
debugger breakpoint |
| :breaklist | :breakl[ist] | list debugger
breakpoints |
| :browse | :bro[wse] | use file
selection dialog |
| :bufdo | :bufd[o] | execute command
in each listed buffer |
| :buffers | :buffers | list all files
in the buffer list |
| :bunload | :bun[load] | unload a
specific buffer |
| :bwipeout | :bw[ipeout] | really delete a
buffer |
| :change | :c[hange] | replace a line
or series of lines |
| :cNext | :cN[ext] | go to previous
error |
| :cNfile | :cNf[ile] | go to last
error in previous file |
| :cabbrev | :ca[bbrev] | like
":abbreviate" but for Command-line mode |
| :cabclear | :cabcl[ear] | clear all
abbreviations for Command-line mode |
| :cabove | :cabo[ve] | go to error
above current line |
| :caddbuffer | :cad[dbuffer] | add errors from
... |

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

buffer
[:caddexpr](#) :cadde[xpr] add errors from
 expr
[:caddfile](#) :caddf[ile] add error
 message to current quickfix list
[:cafter](#) :caf[ter] go to error
 after current cursor
[:call](#) :cal[l] call a function
[:catch](#) :cat[ch] part of a :try
 command
[:cbefore](#) :cbe[fore] go to error
 before current cursor
[:cbelow](#) :cbel[ow] go to error
 below current line
[:cbottom](#) :cbo[ttom] scroll to the
 bottom of the quickfix window
[:cbuffer](#) :cb[u]ffer parse error
 messages and jump to first error
[:cc](#) :cc go to specific
 error
[:cclose](#) :ccl[ose] close quickfix
 window
[:cd](#) :cd change
 directory
[:cdo](#) :cdo execute command
 in each valid error list entry
[:cfdo](#) :cfd[o] execute command
 in each file in error list
[:center](#) :ce[nter] format lines at
 the center
[:cexpr](#) :cex[pr] read errors
 from expr and jump to first
[:cfile](#) :cf[ile] read file with
 error messages and jump to first
[:cfirst](#) :cfir[st] go to the
 specified error, default first one
[:cgetbuffer](#) :cgetb[u]ffer get errors from
 buffer
[:cgetexpr](#) :cgete[xpr] get errors from
 expr

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

| | | |
|------------------------------|----------------|---|
| :cgetfile | :cg[etfile] | read file with
error messages |
| :changes | :changes | print the
change list |
| :chdir | :chd[ir] | change
directory |
| :checkhealth | :che[ckhealth] | run
healthchecks |
| :checkpath | :checkp[ath] | list included
files |
| :checktime | :checkt[ime] | check timestamp
of loaded buffers |
| :chistory | :chi[story] | list the error
lists |
| :clast | :cla[st] | go to the
specified error, default last one |
| :clearjumps | :cle[arjumps] | clear the jump
list |
| :clist | :cl[ist] | list all errors |
| :close | :clo[se] | close current
window |
| :cmap | :cm[ap] | like ":map" but
for Command-line mode |
| :cmapclear | :cmapc[lear] | clear all
mappings for Command-line mode |
| :cmenu | :cme[nu] | add menu for
Command-line mode |
| :cnext | :cn[ext] | go to next
error |
| :cnewer | :cnew[er] | go to newer
error list |
| :cnfile | :cnf[ile] | go to first
error in next file |
| :cnoremap | :cno[remap] | like ":noremap"
but for Command-line mode |
| :cnoreabbrev | :cnorea[bbrev] | like
":noreabbrev" but for Command-line mode |
| :cnoremenu | :cnoreme[nu] | like
":noremenu" but for Command-line mode |
| :copy | :co[py] | copy lines |

[Main](#)
[Commands index](#)
[Quick reference](#)

[1. Insert Mode](#)
[2. Normal Mode](#)
[2.1 Text Objects](#)
[2.2 Window Commands](#)
[2.3 Square Bracket Com](#)
[2.4 Commands Starting](#)
[2.5 Commands Starting](#)
[2.6 Operator-Pending M](#)
[3. Visual Mode](#)
[4. Command-Line Editin](#)
[5. Terminal Mode](#)
[6. Ex Commands](#)

| | | |
|---------------------------------------|--------------------------------|-----------------|
| :colder | :col[der] | go to older |
| error list | | |
| :colorscheme | :colo[rscheme] | load a specific |
| color scheme | | |
| :command | :com[mand] | create user- |
| defined command | | |
| :comclear | :comc[lear] | clear all user- |
| defined commands | | |
| :compiler | :comp[iler] | do settings for |
| a specific compiler | | |
| :continue | :con[tinue] | go back to |
| :while | | |
| :confirm | :conf[irm] | prompt user |
| when confirmation required | | |
| :const | :cons[t] | create a |
| variable as a constant | | |
| :copen | :cope[n] | open quickfix |
| window | | |
| :cprevious | :cp[revious] | go to previous |
| error | | |
| :cpfile | :cpf[ile] | go to last |
| error in previous file | | |
| :cquit | :cq[uit] | quit Vim with |
| an error code | | |
| :crewind | :cr[ewind] | go to the |
| specified error, default first one | | |
| :cunmap | :cu[nmap] | like ":unmap" |
| but for Command-line mode | | |
| :cunabbrev | :cuna[bbrev] | like |
| ":unabbrev" but for Command-line mode | | |
| :cunmenu | :cunme[nu] | remove menu for |
| Command-line mode | | |
| :cwindow | :cw[indow] | open or close |
| quickfix window | | |
| :delete | :d[ele]te | delete lines |
| :debug | :deb[ug] | run a command |
| in debugging mode | | |
| :debuggreedy | :debugg[reedy] | read debug mode |
| commands from normal input | | |
| :defer | :defe[r] | call function |

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

when current function is done

[:delcommand](#) :delc[ommand] delete user-defined command

[:delfunction](#) :delf[unction] delete a user function

[:delmarks](#) :delm[arks] delete marks

[:detach](#) :detach detach the current UI

[:diffupdate](#) :dif[fupdate] update ['diff'](#) buffers

[:diffget](#) :diffg[et] remove differences in current buffer

[:diffoff](#) :diffo[ff] switch off diff mode

[:diffpatch](#) :diffp[atch] apply a patch and show differences

[:diffput](#) :diffpu[t] remove differences in other buffer

[:diffsplit](#) :diffs[plit] show differences with another file

[:diffthis](#) :diff[t[his]] make current window a diff window

[:digraphs](#) :dig[raps] show or enter digraphs

[:display](#) :di[splay] display registers

[:djump](#) :dj[ump] jump to #define

[:dl](#) :dl short for

[:delete](#) with the 'l' flag

[:dlist](#) :dli[st] list #defines

[:doautocmd](#) :do[autocmd] apply autocommands to current buffer

[:doautoall](#) :doautoa[ll] apply autocommands for all loaded buffers

[:dp](#) :d[elete]p short for

[:delete](#) with the 'p' flag

[:drop](#) :dr[op] jump to window editing file or edit file in

[:dsearch](#) :ds[earch] current window list one

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

| | | |
|--------------------------------------|----------------|-----------------|
| #define | | |
| :dsplit | :dsp[lit] | split window |
| and jump to #define | | |
| :edit | :e[dit] | edit a file |
| :earlier | :ea[rlier] | go to older |
| change, undo | | |
| :echo | :ec[ho] | echoes the |
| result of expressions | | |
| :echoerr | :echoe[rr] | like :echo, |
| show like an error and use history | | |
| :echohl | :echoh[l] | set |
| highlighting for echo commands | | |
| :echomsg | :echom[sg] | same as :echo, |
| put message in history | | |
| :echon | :echon | same as :echo, |
| but without <code><EOL></code> | | |
| :else | :el[se] | part of an :if |
| command | | |
| :elseif | :elsei[f] | part of an :if |
| command | | |
| :emenu | :em[enu] | execute a menu |
| by name | | |
| :endif | :en[dif] | end previous |
| :if | | |
| :endfor | :endfo[r] | end previous |
| :for | | |
| :endfunction | :endf[unction] | end of a user |
| function started with :function | | |
| :endtry | :endt[ry] | end previous |
| :try | | |
| :endwhile | :endw[hile] | end previous |
| :while | | |
| :enew | :ene[w] | edit a new, |
| unnamed buffer | | |
| :eval | :ev[al] | evaluate an |
| expression and discard the result | | |
| :ex | :ex | same as ":edit" |
| :execute | :exe[cute] | execute result |
| of expressions | | |
| :exit | :exi[t] | same as ":xit" |

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

| | | | |
|-------------------------------|---------------------------------|--|--|
| :exusage | :exu[sage] | overview of Ex commands | Main |
| :fclose | :fc[lose] | close floating window | Commands index |
| :file | :f[ile] | show or set the current file name | Quick reference |
| :files | :files | list all files in the buffer list | <hr/> |
| :filetype | :filet[ype] | switch file type detection on/off | 1. Insert Mode |
| :filter | :filt[er] | filter output of following command | 2. Normal Mode |
| :find | :fin[d] | find file in <code>'path'</code> and edit it | 2.1 Text Objects |
| :finally | :fina[lly] | part of a <code>:try</code> command | 2.2 Window Commands |
| :finish | :fini[sh] | quit sourcing a Vim script | 2.3 Square Bracket Com |
| :first | :fir[st] | go to the first file in the argument list | 2.4 Commands Starting |
| :fold | :fo[ld] | create a fold | 2.5 Commands Starting |
| :foldclose | :foldc[lose] | close folds | 2.6 Operator-Pending M |
| :foldddopen | :folddd[oopen] | execute command on lines not in a closed fold | 3. Visual Mode |
| :folddoclosed | :folddoc[losed] | execute command on lines in a closed fold | 4. Command-Line Editin |
| :foldopen | :foldo[pen] | open folds | 5. Terminal Mode |
| :for | :for | for loop | 6. Ex Commands |
| :function | :fu[nction] | define a user function | |
| :global | :g[lobal] | execute commands for matching lines | |
| :goto | :go[to] | go to byte in the buffer | |
| :grep | :gr[ep] | run <code>'grepprg'</code> and jump to first match | |
| :grepadd | :grepa[dd] | like <code>:grep</code> , but append to current list | |
| :gui | :gu[i] | start the GUI | |
| :gvim | :gv[im] | start the GUI | |

| | | |
|------------------------------|--------------------------------|---|
| :help | :h[elp] | open a help window |
| :helpclose | :helpc[lose] | close one help window |
| :helpgrep | :helpg[rep] | like <code>":grep"</code> but searches help files |
| :helptags | :helpt[ags] | generate help tags for a directory |
| :highlight | :hi[ghlight] | specify highlighting methods |
| :hide | :hid[e] | hide current buffer for a command |
| :history | :his[tory] | print a history list |
| :horizontal | :hor[izontal] | following window command work horizontally |
| :insert | :i[nsert] | insert text |
| :iabbrev | :ia[bbrev] | like <code>":abbrev"</code> but for Insert mode |
| :iabclear | :iabc[lear] | like <code>":abclear"</code> but for Insert mode |
| :if | :if | execute commands when condition met |
| :ijump | :ij[ump] | jump to definition of identifier |
| :ilist | :il[ist] | list lines where identifier matches |
| :imap | :im[ap] | like <code>":map"</code> but for Insert mode |
| :imapclear | :imapc[lear] | like <code>":mapclear"</code> but for Insert mode |
| :imenu | :ime[nu] | add menu for Insert mode |
| :inoremap | :ino[remap] | like <code>":noremap"</code> but for Insert mode |
| :inoreabbrev | :inorea[bbrev] | like <code>":noreabbrev"</code> but for Insert mode |
| :inoremenu | :inoreme[nu] | like <code>":noremenu"</code> but for Insert mode |
| :intro | :int[ro] | print the |

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

introductory message

[:input](#) :ip[ut] like [:put](#), but
adjust the indent to the

current line
list one line

[:isearch](#) :is[earch]

where identifier matches

[:isplit](#) :isp[lit]

and jump to definition of

identifier
like ":unmap"

[:iunmap](#) :iu[nmap]

but for Insert mode

[:iunabbrev](#) :iuna[bbrev] like

":unabbrev" but for Insert mode

[:iunmenu](#) :iunme[nu] remove menu for

Insert mode

[:join](#) :j[oin] join lines

[:jumps](#) :ju[mps] print the jump

list

[:k](#) :k set a mark

[:keepalt](#) :keepa[lt] following

command keeps the alternate file

[:keepmarks](#) :kee[pmarks] following

command keeps marks where they are

[:keepjumps](#) :keepj[umps] following

command keeps jumplist and marks

[:keeppatterns](#) :keepp[atterns] following

command keeps search pattern history

[:lNext](#) :lN[ext] go to previous

entry in location list

[:lNfile](#) :lNf[ile] go to last

entry in previous file

[:list](#) :l[ist] print lines

[:labove](#) :lab[ove] go to location

above current line

[:laddexpr](#) :lad[dexpr] add locations

from expr

[:laddbuffer](#) :laddb[uffer] add locations

from buffer

[:laddfile](#) :laddf[ile] add locations

to current location list

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

| | | |
|-----------------------------|---------------|---|
| :lafter | :laf[ter] | go to location after current cursor |
| :last | :la[st] | go to the last file in the argument list |
| :language | :lan[guage] | set the language (locale) |
| :later | :lat[er] | go to newer change, redo |
| :lbefore | :lbe[fore] | go to location before current cursor |
| :lbelow | :lbel[ow] | go to location below current line |
| :lbottom | :lbo[ttom] | scroll to the bottom of the location window |
| :lbuffer | :lb[uffer] | parse locations and jump to first location |
| :lcd | :lc[d] | change directory locally |
| :lchdir | :lch[dir] | change directory locally |
| :lclose | :lcl[ose] | close location window |
| :ldo | :ld[o] | execute command in valid location list entries |
| :lfdo | :lfd[o] | execute command in each file in location list |
| :left | :le[ft] | left align lines |
| :leftabove | :lefta[bove] | make split window appear left or above |
| :let | :let | assign a value to a variable or option |
| :lexpr | :lex[pr] | read locations from expr and jump to first |
| :lfile | :lf[ile] | read file with locations and jump to first |
| :lfirst | :lfir[st] | go to the specified location, default first one |
| :lgetbuffer | :lgetb[uffer] | get locations from buffer |

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

| | | |
|-----------------------------|-------------------------------|---|
| :lgetexpr | :lgete[xpr] | get locations from expr |
| :lgetfile | :lg[etfile] | read file with locations |
| :lgrep | :lgr[ep] | run 'grepprg' and jump to first match |
| :lgrepadd | :lgrepa[dd] | like :grep , but append to current list |
| :lhelpgrep | :lh[elpgrep] | like ":helpgrep" but uses location list |
| :lhistory | :lhi[story] | list the location lists |
| :ll | :ll | go to specific location |
| :llast | :lla[st] | go to the specified location, default last one |
| :llist | :lli[st] | list all locations |
| :lmake | :lmak[e] | execute external command 'makeprg' and parse error messages |
| :lmap | :lm[ap] | like ":map!" but includes Lang-Arg mode |
| :lmapclear | :lmapc[lear] | like ":mapclear!" but includes Lang-Arg mode |
| :lnext | :lne[xt] | go to next location |
| :lnewer | :lnew[er] | go to newer location list |
| :lnfile | :lnf[ile] | go to first location in next file |
| :lnoremap | :ln[oremap] | like ":noremap!" but includes Lang-Arg mode |
| :loadkeymap | :loadk[eymap] | load the following keymaps until EOF |
| :loadview | :lo[adview] | load view for current window from a file |
| :lockmarks | :loc[kmarks] | following command keeps marks where they are |
| :lockvar | :lockv[ar] | lock variables |

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

| | | | |
|--|--------------------------------|-----------------------------|--|
| :log | :log | view a log file | Main |
| :lolder | :lol[der] | go to older | Commands index |
| location list | | | Quick reference |
| :lopen | :lop[en] | open location | |
| window | | | |
| :lprevious | :lp[revious] | go to previous | 1. Insert Mode |
| location | | | 2. Normal Mode |
| :lpfile | :lpf[ile] | go to last | 2.1 Text Objects |
| location in previous file | | | 2.2 Window Commands |
| :lrewind | :lr[ewind] | go to the | 2.3 Square Bracket Com |
| specified location, default first one | | | 2.4 Commands Starting |
| :ls | :ls | list all | 2.5 Commands Starting |
| buffers | | | 2.6 Operator-Pending M |
| :lsp | :lsp | language server | 3. Visual Mode |
| protocol | | | 4. Command-Line Editin |
| :ltag | :lt[ag] | jump to tag and | 5. Terminal Mode |
| add matching tags to the | | | 6. Ex Commands |
| | | location list | |
| :lunmap | :lu[nmap] | like ":unmap!" | |
| but includes Lang-Arg mode | | | |
| :lua | :lua | execute Lua | |
| command | | | |
| :luado | :luad[o] | execute Lua | |
| command for each line | | | |
| :luafile | :luaf[ile] | execute Lua | |
| script file | | | |
| :lvimgrep | :lv[imgrep] | search for | |
| pattern in files | | | |
| :lvimgrepadd | :lvimgrepa[dd] | like :vimgrep, | |
| but append to current list | | | |
| :lwindow | :lw[indow] | open or close | |
| location window | | | |
| :move | :m[ove] | move lines | |
| :mark | :ma[rk] | set a mark | |
| :make | :mak[e] | execute | |
| external command 'makeprg' and parse | | | |
| | | error messages | |
| :map | :map | show or enter a | |
| mapping | | | |
| :mapclear | :mapc[lear] | clear all | |

mappings for Normal and Visual mode

| | | |
|--------------------------------|------------------|---|
| :marks | :marks | list all marks |
| :match | :mat[ch] | define a match to highlight |
| :menu | :me[nu] | enter a new menu item |
| :menutranslate | :menut[ranslate] | add a menu translation item |
| :messages | :mes[sages] | view previously displayed messages |
| :mkexrc | :mk[exrc] | write current mappings and settings to a file |
| :mksession | :mks[ession] | write session info to a file |
| :mkspell | :mksp[ell] | produce .spl spell file |
| :mkvimrc | :mkv[imrc] | write current mappings and settings to a file |
| :mkview | :mkvie[w] | write view of current window to a file |
| :mode | :mod[e] | show or change the screen mode |
| :next | :n[ext] | go to next file in the argument list |
| :new | :new | create a new empty window |
| :nmap | :nm[ap] | like ":map" but for Normal mode |
| :nmapclear | :nmapc[lear] | clear all mappings for Normal mode |
| :nmenu | :nme[nu] | add menu for Normal mode |
| :nnoremap | :nn[oremap] | like ":noremap" but for Normal mode |
| :nnoremenu | :nnoreme[nu] | like ":noremenu" but for Normal mode |
| :noautocmd | :noa[utocmd] | following commands don't trigger autocommands |
| :noremap | :no[remap] | enter a mapping that will not be remapped |

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

| | | |
|--|---------------|---------------------------------|
| :nohlsearch | :noh[lsearch] | suspend |
| 'hlsearch' | highlighting | |
| :noreabbrev | :norea[bbrev] | enter an |
| abbreviation that will not be | | |
| | | remapped |
| :noremenu | :noreme[nu] | enter a menu |
| that will not be remapped | | |
| :normal | :norm[al] | execute Normal |
| mode commands | | |
| :noswapfile | :nos[wapfile] | following |
| commands don't create a swap file | | |
| :number | :nu[mber] | print lines |
| with line number | | |
| :nunmap | :nun[map] | like ":unmap" |
| but for Normal mode | | |
| :nunmenu | :nunme[nu] | remove menu for |
| Normal mode | | |
| :oldfiles | :ol[dfiles] | list files that |
| have marks in the shada file | | |
| :omap | :om[ap] | like ":map" but |
| for Operator-pending mode | | |
| :omapclear | :omapc[lear] | remove all |
| mappings for Operator-pending mode | | |
| :omenu | :ome[nu] | add menu for |
| Operator-pending mode | | |
| :only | :on[ly] | close all |
| windows except the current one | | |
| :onoremap | :ono[remap] | like ":noremap" |
| but for Operator-pending mode | | |
| :onoremenu | :onoreme[nu] | like |
| ":noremenu" but for Operator-pending mode | | |
| :options | :opt[ions] | open the |
| options-window | | |
| :ounmap | :ou[nmap] | like ":unmap" |
| but for Operator-pending mode | | |
| :ounmenu | :ounme[nu] | remove menu for |
| Operator-pending mode | | |
| :packadd | :pa[ckadd] | add a plugin |
| from 'packpath' | | |
| :packdel | :packd[el] | remove vim.pack |

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

| | | | |
|---|----------------|---------------------------------|--|
| plugins | | | Main |
| :packloadall | :packl[oadall] | load all | Commands index |
| packages under ' packpath ' | | | Quick reference |
| :packupdate | :packu[pdate] | update vim.pack | |
| plugins | | | |
| :pbuffer | :pb[uffer] | edit buffer in | 1. Insert Mode |
| the preview window | | | 2. Normal Mode |
| :pclose | :pc[lose] | close preview | 2.1 Text Objects |
| window | | | 2.2 Window Commands |
| :pedit | :ped[it] | edit file in | 2.3 Square Bracket Com |
| the preview window | | | 2.4 Commands Starting |
| :perl | :pe[rl] | execute perl | 2.5 Commands Starting |
| command | | | 2.6 Operator-Pending M |
| :perldo | :perld[o] | execute perl | 3. Visual Mode |
| command for each line | | | 4. Command-Line Editin |
| :perlfile | :perl[ile] | execute perl | 5. Terminal Mode |
| script file | | | 6. Ex Commands |
| :print | :p[rint] | print lines | |
| :profdel | :profd[el] | stop profiling | |
| a function or script | | | |
| :profile | :prof[ile] | profiling | |
| functions and scripts | | | |
| :pop | :po[p] | jump to older | |
| entry in tag stack | | | |
| :popup | :popu[p] | popup a menu by | |
| name | | | |
| :ppop | :pp[op] | ":pop" in | |
| preview window | | | |
| :preserve | :pre[serve] | write all text | |
| to swap file | | | |
| :previous | :prev[ious] | go to previous | |
| file in argument list | | | |
| :psearch | :ps[earch] | like "":ijump" | |
| but shows match in preview window | | | |
| :ptag | :pt[ag] | show tag in | |
| preview window | | | |
| :ptNext | :ptN[ext] | :tNext in | |
| preview window | | | |
| :ptfirst | :ptf[irst] | :trewind in | |
| preview window | | | |

| | | |
|--|-------------------------------|---------------------------------|
| :ptjump | :ptj[ump] | :tjump and show |
| tag in preview window | | |
| :ptlast | :ptl[ast] | :tlast in |
| preview window | | |
| :ptnext | :ptn[ext] | :tnext in |
| preview window | | |
| :ptprevious | :ptp[revious] | :tprevious in |
| preview window | | |
| :ptrewind | :ptr[ewind] | :trewind in |
| preview window | | |
| :ptselect | :pts[elect] | :tselect and |
| show tag in preview window | | |
| :put | :pu[t] | insert contents |
| of register in the text | | |
| :pwd | :pw[d] | print current |
| directory | | |
| :py3 | :py3 | execute Python |
| 3 command | | |
| :python3 | :python3 | same as :py3 |
| :py3do | :py3d[o] | execute Python |
| 3 command for each line | | |
| :py3file | :py3f[ile] | execute Python |
| 3 script file | | |
| :python | :py[thon] | execute Python |
| command | | |
| :pydo | :pyd[o] | execute Python |
| command for each line | | |
| :pyfile | :pyf[ile] | execute Python |
| script file | | |
| :pyx | :pyx | execute |
| python_x command | | |
| :pythonx | :pythonx | same as :pyx |
| :pyxdo | :pyxd[o] | execute |
| python_x command for each line | | |
| :pyxfile | :pyx f[ile] | execute |
| python_x script file | | |
| :quit | :q[uit] | quit current |
| window (when one window quit Vim) | | |
| :quitall | :quita[ll] | quit Vim |
| :qall | :qa[ll] | quit Vim |

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

| | | |
|--------------------------------|----------------------------------|--|
| :read | :r[ead] | read file into the text |
| :recover | :rec[over] | recover a file from a swap file |
| :redo | :red[o] | redo one undone change |
| :redir | :redi[r] | redirect messages to a file or register |
| :redraw | :redr[aw] | force a redraw of the display |
| :redrawstatus | :redraws[tatus] | force a redraw of the status line(s) and window bar(s) |
| :redrawtabline | :redrawt[abline] | force a redraw of the tabline |
| :registers | :reg[isters] | display the contents of registers |
| :resize | :res[ize] | change current window height |
| :restart | :restart | restart the Nvim server |
| :retab | :ret[ab] | change tab size |
| :return | :retu[rn] | return from a user function |
| :rewind | :rew[ind] | go to the first file in the argument list |
| :right | :ri[ght] | right align text |
| :rightbelow | :rightb[elow] | make split window appear right or below |
| :rshada | :rsh[ada] | read from shada file |
| :ruby | :rub[y] | execute Ruby command |
| :rubydo | :rubyd[o] | execute Ruby command for each line |
| :rubyfile | :rubyf[ile] | execute Ruby script file |
| :rundo | :rund[o] | read undo information from a file |

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

| | | |
|--|---------------|-----------------|
| :runtime | :ru[ntime] | source vim |
| scripts in ' runtimepath ' | | |
| :substitute | :s[ubstitute] | find and |
| replace text | | |
| :sNext | :sN[ext] | split window |
| and go to previous file in | | |
| | | argument list |
| :sandbox | :san[dbox] | execute a |
| command in the sandbox | | |
| :sargument | :sa[rgument] | split window |
| and go to specific file in | | |
| | | argument list |
| :sall | :sal[l] | open a window |
| for each file in argument list | | |
| :saveas | :sav[eas] | save file under |
| another name. | | |
| :sbuffer | :sb[uffer] | split window |
| and go to specific file in the | | |
| | | buffer list |
| :sbNext | :sbN[ext] | split window |
| and go to previous file in the | | |
| | | buffer list |
| :sball | :sba[ll] | open a window |
| for each file in the buffer list | | |
| :sbfirst | :sbf[irst] | split window |
| and go to first file in the | | |
| | | buffer list |
| :sblast | :sbl[ast] | split window |
| and go to last file in buffer | | |
| | | list |
| :sbmodified | :sbm[odified] | split window |
| and go to modified file in the | | |
| | | buffer list |
| :sbnext | :sbn[ext] | split window |
| and go to next file in the buffer | | |
| | | list |
| :sbprevious | :sbp[revious] | split window |
| and go to previous file in the | | |
| | | buffer list |
| :sbrewind | :sbr[ewind] | split window |

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

and go to first file in the

| | | |
|---------------------------------|-------------------|---|
| :scriptnames | :scr[iptnames] | buffer list
list names of
all sourced Vim scripts |
| :scriptencoding | :scripte[ncoding] | encoding
used in sourced Vim script |
| :set | :se[t] | show or set
options |
| :setfiletype | :setf[iletype] | set 'filetype' ,
unless it was set already |
| :setglobal | :setg[lobal] | show global
values of options |
| :setlocal | :setl[ocal] | show or set
options locally |
| :sfind | :sf[ind] | split current
window and edit file in 'path' |
| :sfirst | :sfir[st] | split window
and go to first file in the |
| :sign | :sig[n] | argument list
manipulate
signs |
| :silent | :sil[ent] | run a command
silently |
| :sleep | :sl[eepest] | do nothing for
a few seconds |
| :sleep! | :sl[eepest]! | do nothing for
a few seconds, without the |
| :slast | :sla[st] | cursor visible
split window
and go to last file in the |
| :smagic | :sm[agic] | argument list
:substitute
with 'magic' |
| :smap | :smap | like <code>":map"</code> but
for Select mode |
| :smapclear | :smapc[lear] | remove all
mappings for Select mode |
| :smenu | :sme[nu] | add menu for
Select mode |
| :snext | :sn[ext] | split window |

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

and go to next file in the

[:snomagic](#) :sno[magic]

with '[nomagic](#)'

[:snoremap](#) :snor[emap]

but for Select mode

[:snoremenu](#) :snoreme[nu]

":noremenu" but for Select mode

[:sort](#) :sor[t]

[:source](#) :so[urce]

commands from a file

[:spelldump](#) :spelld[ump]

and fill with all correct words

[:spellgood](#) :spe[llgood]

for spelling

[:spellinfo](#) :spelli[nfo]

loaded spell files

[:spellrare](#) :spellra[re]

for spelling

[:spellrepall](#) :spellr[epall]

words like last [z=](#)

[:spellundo](#) :spellu[ndo]

bad word

[:spellwrong](#) :spellw[rong]

mistake

[:split](#) :sp[lit]

window

[:sprevious](#) :spr[evious]

and go to previous file in the

[:srewind](#) :sre[wind]

and go to first file in the

[:stop](#) :st[op]

editor or escape to a shell

[:stag](#) :sta[g]

and jump to a tag

[:startinsert](#) :star[tinsert]

mode

[:startgreplace](#) :startg[replace]

argument list

:substitute

like ":noremap"

like

sort lines

read Vim or Ex

split window

add good word

show info about

add rare word

replace all bad

remove good or

add spelling

split current

split window

argument list

split window

argument list

suspend the

split window

start Insert

start Virtual

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

Replace mode
[:startreplace](#) :startr[eplace] start Replace mode
[:stopinsert](#) :stopi[nsert] stop Insert mode
[:stjump](#) :stj[ump] do ":tjump" and split window
[:stselect](#) :sts[elect] do ":tselect" and split window
[:sunhide](#) :sun[hide] same as ":unhide"
[:sunmap](#) :sunm[ap] like ":unmap" but for Select mode
[:sunmenu](#) :sunme[nu] remove menu for Select mode
[:suspend](#) :sus[pend] same as ":stop"
[:svview](#) :sv[iew] split window and edit file read-only
[:swapname](#) :sw[apname] show the name of the current swap file
[:syntax](#) :sy[ntax] syntax highlighting
[:syntime](#) :synti[me] measure syntax highlighting speed
[:syncbind](#) :sync[bind] sync scroll binding
[:t](#) :t same as ":copy"
[:tNext](#) :tN[ext] jump to previous matching tag
[:tabNext](#) :tabN[ext] go to previous tabpage
[:tabclose](#) :tabc[lose] close current tabpage
[:tabdo](#) :tabd[o] execute command in each tabpage
[:tabedit](#) :tabe[dit] edit a file in a new tabpage
[:tabfind](#) :tabf[ind] find file in 'path', edit it in a new tabpage
[:tabfirst](#) :tabfir[st] go to first

[Main](#)
[Commands index](#)
[Quick reference](#)

[1. Insert Mode](#)
[2. Normal Mode](#)
[2.1 Text Objects](#)
[2.2 Window Commands](#)
[2.3 Square Bracket Com](#)
[2.4 Commands Starting](#)
[2.5 Commands Starting](#)
[2.6 Operator-Pending M](#)
[3. Visual Mode](#)
[4. Command-Line Editin](#)
[5. Terminal Mode](#)
[6. Ex Commands](#)

| | | | |
|--|----------------|-----------------|--|
| tabpage | | | Main |
| :tablast | :tabl[ast] | go to last | Commands index |
| tabpage | | | Quick reference |
| :tabmove | :tabm[ove] | move tabpage to | |
| other position | | | |
| :tabnew | :tabnew | edit a file in | 1. Insert Mode |
| a new tabpage | | | 2. Normal Mode |
| :tabnext | :tabn[ext] | go to next | 2.1 Text Objects |
| tabpage | | | 2.2 Window Commands |
| :tabonly | :tabo[nly] | close all | 2.3 Square Bracket Com |
| tabpages except the current one | | | 2.4 Commands Starting |
| :tabprevious | :tabp[revious] | go to previous | 2.5 Commands Starting |
| :tabrewind | :tabr[ewind] | go to first | 2.6 Operator-Pending M |
| :tabs | :tabs | list the | 3. Visual Mode |
| tabpages and what they contain | | | 4. Command-Line Editin |
| :tab | :tab | create new tab | 5. Terminal Mode |
| when opening new window | | | 6. Ex Commands |
| :tag | :ta[g] | jump to tag | |
| :tags | :tags | show the | |
| contents of the tag stack | | | |
| :tcd | :tc[d] | change | |
| directory for tabpage | | | |
| :tchdir | :tch[dir] | change | |
| directory for tabpage | | | |
| :terminal | :te[rminal] | open a terminal | |
| buffer | | | |
| :tfirst | :tf[irst] | jump to first | |
| matching tag | | | |
| :throw | :th[row] | throw an | |
| exception | | | |
| :tjump | :tj[ump] | like | |
| ":tselect", but jump directly when there | | is only one | |
| match | | | |
| :tlast | :tl[ast] | jump to last | |
| matching tag | | | |
| :tmenu | :tlm[enu] | add menu for | |
| Terminal-mode | | | |

| | | |
|-------------------------------|---------------------------------|---|
| :tlnoremenu | :tln[oremenu] | like
":noremenu" but for Terminal-mode |
| :tlunmenu | :tlu[nmenu] | remove menu for
Terminal-mode |
| :tmapclear | :tmapc[lear] | remove all
mappings for Terminal-mode |
| :tmap | :tma[p] | like ":map" but
for Terminal-mode |
| :tmenu | :tm[enu] | define menu
tooltip |
| :tnext | :tn[ext] | jump to next
matching tag |
| :tnoremap | :tno[remap] | like ":noremap"
but for Terminal-mode |
| :topleft | :to[pleft] | make split
window appear at top or far left |
| :tprevious | :tp[revious] | jump to
previous matching tag |
| :trewind | :tr[ewind] | jump to first
matching tag |
| :trust | :trust | add or remove
file from trust database |
| :try | :try | execute
commands, abort on error or exception |
| :tselect | :ts[elect] | list matching
tags and select one |
| :tunmap | :tunma[p] | like ":unmap"
but for Terminal-mode |
| :tunmenu | :tu[nmenu] | remove menu
tooltip |
| :undo | :u[ndo] | undo last
change(s) |
| :undojoin | :undoj[oin] | join next
change with previous undo block |
| :undolist | :undol[ist] | list leafs of
the undo tree |
| :unabbreviate | :una[bbreviate] | remove
abbreviation |
| :unhide | :unh[ide] | open a window
for each loaded file in the |

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

| | | | |
|--------------------------------------|-------------------------------|-----------------|--|
| :uniq | :uni[q] | buffer list | Main |
| :unlet | :unl[et] | uniq lines | Commands index |
| :unlockvar | :unlo[ckvar] | delete variable | Quick reference |
| variables | | unlock | |
| :unmap | :unm[ap] | remove mapping | 1. Insert Mode |
| :unmenu | :unme[nu] | remove menu | 2. Normal Mode |
| :unsilent | :uns[ilent] | run a command | 2.1 Text Objects |
| not silently | | | 2.2 Window Commands |
| :update | :up[date] | write buffer if | 2.3 Square Bracket Com |
| modified | | | 2.4 Commands Starting |
| :uptime | :upt[ime] | show Nvim's | 2.5 Commands Starting |
| uptime | | | 2.6 Operator-Pending M |
| :vglobal | :v[global] | execute | 3. Visual Mode |
| commands for not matching lines | | | 4. Command-Line Editin |
| :version | :ve[rsion] | print version | 5. Terminal Mode |
| number and other info | | | 6. Ex Commands |
| :verbose | :verb[ose] | execute command | |
| with ' verbose ' set | | | |
| :vertical | :vert[ical] | make following | |
| command split vertically | | | |
| :vimgrep | :vim[grep] | search for | |
| pattern in files | | | |
| :vimgrepadd | :vimgrepa[dd] | like :vimgrep, | |
| but append to current list | | | |
| :visual | :vi[sual] | same as | |
| ":edit", but turns off "Ex" mode | | | |
| :viusage | :viu[sage] | overview of | |
| Normal mode commands | | | |
| :view | :vie[w] | edit a file | |
| read-only | | | |
| :vmap | :vm[ap] | like ":map" but | |
| for Visual+Select mode | | | |
| :vmapclear | :vmapc[lear] | remove all | |
| mappings for Visual+Select mode | | | |
| :vmenu | :vme[nu] | add menu for | |
| Visual+Select mode | | | |
| :vnew | :vne[w] | create a new | |
| empty window, vertically split | | | |
| :vnoremap | :vn[oremap] | like ":noremap" | |

but for Visual+Select mode
[:vnoremenu](#) :vnoreme[nu] like
 ":noremenu" but for Visual+Select mode
[:vsplit](#) :vs[plit] split current
 window vertically
[:vunmap](#) :vu[nmap] like ":unmap"
 but for Visual+Select mode
[:vunmenu](#) :vunme[nu] remove menu for
 Visual+Select mode
[:windo](#) :wind[o] execute command
 in each window
[:write](#) :w[rite] write to a file
[:wNext](#) :wN[ext] write to a file
 and go to previous file in

[:wall](#) :wa[ll] argument list
 (changed) buffers write all
[:while](#) :wh[ile] execute loop
 for as long as condition met
[:winsize](#) :wi[nsize] get or set
 window size (obsolete)
[:wincmd](#) :winc[md] execute a
 Window (**CTRL-W**) command
[:winpos](#) :winp[os] get or set
 window position
[:wnext](#) :wn[ext] write to a file
 and go to next file in

[:wprevious](#) :wp[revious] argument list
 and go to previous file in write to a file

[:wq](#) :wq argument list
 and quit window or Vim write to a file
[:wqall](#) :wqa[ll] write all
 changed buffers and quit Vim
[:wshada](#) :wsh[ada] write to ShaDa
 file
[:wundo](#) :wu[ndo] write undo
 information to a file
[:xit](#) :x[it] write if buffer

[Main](#)[Commands index](#)[Quick reference](#)[1. Insert Mode](#)[2. Normal Mode](#)[2.1 Text Objects](#)[2.2 Window Commands](#)[2.3 Square Bracket Com](#)[2.4 Commands Starting](#)[2.5 Commands Starting](#)[2.6 Operator-Pending M](#)[3. Visual Mode](#)[4. Command-Line Editin](#)[5. Terminal Mode](#)[6. Ex Commands](#)

changed and close window

[:xall](#) :xa[ll]

same as

[":wqall"](#)

[:xmapclear](#) :xmapc[lear]

remove all

mappings for Visual mode

[:xmap](#) :xm[ap]

like [":map"](#) but

for Visual mode

[:xmenu](#) :xme[nu]

add menu for

Visual mode

[:xnoremap](#) :xn[oremap]

like [":noremap"](#)

but for Visual mode

[:xnoremenu](#) :xnoreme[nu]

like

[":noremenu"](#) but for Visual mode

[:xunmap](#) :xu[nmap]

like [":unmap"](#)

but for Visual mode

[:xunmenu](#) :xunme[nu]

remove menu for

Visual mode

[:yank](#) :y[ank]

yank lines into

a register

[:z](#) :z

print some

lines

[:~](#) :~

repeat last

[":substitute"](#)

[Main](#)

[Commands index](#)

[Quick reference](#)

[1. Insert Mode](#)

[2. Normal Mode](#)

[2.1 Text Objects](#)

[2.2 Window Commands](#)

[2.3 Square Bracket Com](#)

[2.4 Commands Starting](#)

[2.5 Commands Starting](#)

[2.6 Operator-Pending M](#)

[3. Visual Mode](#)

[4. Command-Line Editin](#)

[5. Terminal Mode](#)

[6. Ex Commands](#)

Generated at 2026-05-24 07:52

from [61eddb3](#) 

parse_errors: 0 ([report docs bug...](#)) |

noise_lines: 3